

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Faculty of Computer Science and Engineering

Lecture Notes

Advanced System Architectures

CO5059

Instructor: Tran Ngoc Thinh

Notes by: Phan Nhat Minh (minh.phannhat95@hcmut.edu.vn)

Semester: 261

Quantitative design and evaluation of computer systems: performance and cost, instruction set architectures, pipelining and its hazards, memory hierarchy, multiprocessors, superscalar/VLIW processors, and AI & security chips.

Last updated: October 3, 2026

Contents

1 Lecture 1: Introduction to Advanced System Architectures **1**
- 1.1 Overview . . . 1
- 1.2 Course Organization . . . 1
- 1.3 What is Computer Architecture? . . . 1
- 1.4 A Brief History . . . 3
- 1.5 The Computer Design Cycle . . . 6
- 1.6 Performance and Cost . . . 6
- 1.7 Metrics of Performance . . . 9
- 1.8 Comparing Performance and Speedup . . . 10
- 1.9 Amdahl's Law . . . 12
- 1.10 Marketing Metrics: MIPS and MFLOPS . . . 14
- 1.11 Benchmarks . . . 15
- 1.12 Summarizing Performance . . . 16
- 1.13 Summary . . . 17

2 Lecture 2: Review of Instruction Set Architecture **18**
- 2.1 Overview . . . 18
- 2.2 The Instruction Set . . . 18
- 2.3 ISA Styles . . . 18
- 2.4 Instruction Set Design . . . 21
- 2.5 Addressing Modes . . . 22
- 2.6 Operand Types and Sizes . . . 24
- 2.7 Instruction Format . . . 25
- 2.8 CISC vs. RISC . . . 25
- 2.9 Case Study: MIPS . . . 27
- 2.10 Exercises . . . 36
- 2.11 Summary . . . 37

3 Lecture 3: Pipelining **38**
- 3.1 Overview . . . 38
- 3.2 What is Pipelining? . . . 38
- 3.3 Pipelining Example: Laundry . . . 39
- 3.4 CPU Pipelining . . . 40
- 3.5 MIPS In-Order Single-Issue Integer Pipeline . . . 42
- 3.6 Pipeline Hazards . . . 43
- 3.7 Structural Hazards . . . 44
- 3.8 Data Hazards . . . 45
- 3.9 Minimizing Data Hazard Stalls by Forwarding . . . 47
- 3.10 Data Hazard Even With Forwarding: the Load-Use Hazard . . . 48
- 3.11 Control Hazards . . . 50
- 3.12 Four Branch Hazard Alternatives . . . 51
- 3.13 Exercises . . . 54
- 3.14 Summary . . . 55

1 Lecture 1: Introduction to Advanced System Architectures

September 13, 2026

1.1 Overview

This lecture sets up the vocabulary for the rest of the course: what computer architecture is, how technology trends (Moore's Law, the power wall) pushed the industry from fast uniprocessors to multicore chips, how the cost of a chip is determined, and—most importantly—how to *measure and compare performance* using execution time, the CPU equation, and Amdahl's Law.

1.2 Course Organization

1.2.1 Administrative issues

- **Textbook:** J. Hennessy, D. Patterson, *Computer Architecture: A Quantitative Approach*, 5th ed., Morgan Kaufmann, 2012 (main reference); W. Stallings, *Computer Organization and Architecture*, 7th ed.; K. Hwang, *Advanced System Architectures: Parallelism, Scalability, Programmability*; plus IEEE/ACM research papers.
- **Grades:** 20% homework + quizzes, 30% assignments + presentations, 50% final exam.

1.2.2 Course coverage

- **Introduction** – brief history of computers, basic concepts of computer system architectures.
- **Instruction Set Principles** – classifying ISAs, addressing modes, operand types/sizes, operations, control-flow instructions, instruction format, the role of compilers.
- **Pipelining** – basic and intermediate concepts, organization of pipelined units, pipeline hazards, reducing branch penalties, branch prediction.
- **Memory Hierarchy Design** – memory hierarchy, cache memories, virtual memories, memory management.
- **Multiprocessors** – classification and performance of parallel architectures, cache coherence.
- **Superscalar & VLIW** – superscalar vs. superpipelined design, instruction dependences, instruction-level parallelism, VLIW.
- **AI & Security chips.**

1.2.3 Prerequisites

Computer Organization & Architecture (combinational/sequential logic, processor, memory, assembly language); Data Structures/Algorithms (complexity analysis, efficient implementations); Operating Systems (task scheduling, management of processors, memory, I/O devices).

1.3 What is Computer Architecture?

1.3.1 A changing definition

- **1950s–60s:** Computer Arithmetic.
- **1970s–mid-80s:** Instruction Set Design.
- **1990s:** Design of CPU, memory system, I/O system, multiprocessors, networks.

- **2000s:** Multi-core design, on-chip networking, parallel programming paradigms, power reduction.
- **2010s:** Low-power/mobile processing, self-adapting systems.
- **2020s:** AI chips, quantum computing.

1.3.2 Role of a computer architect

Definition 1.1 (Role of a computer architect). To design and engineer the various levels of a computer system to maximize **performance** and **programmability** within limits of **technology** and **cost**.

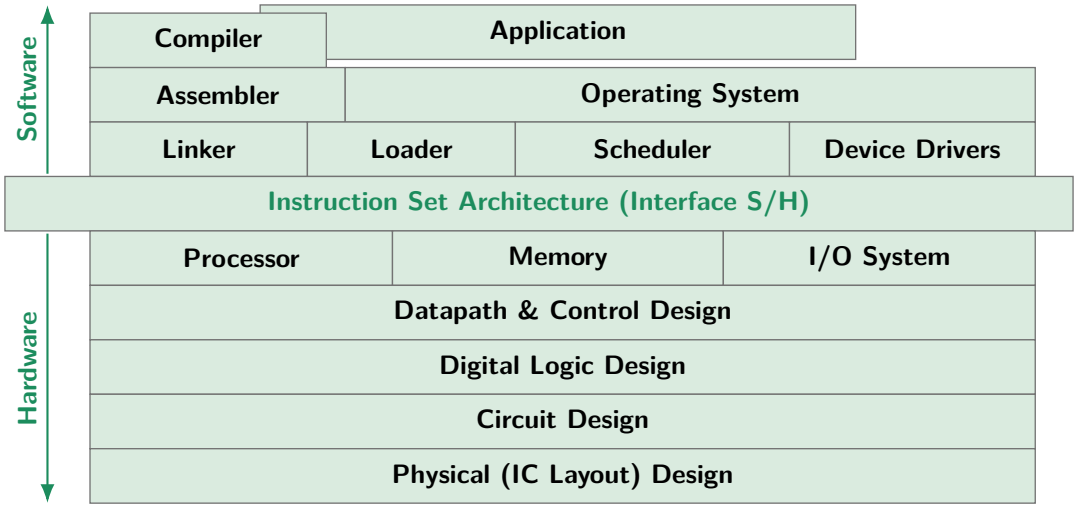
1.3.3 The task of the computer designer

- Determine what attributes are important for a new machine.
- Design a machine to maximize cost-performance.
- What are these tasks?
  - Instruction set design
  - Functional organization
  - Logic design
  - Implementation: IC design, packaging, power, cooling, …

1.3.4 Levels of abstraction

Software and hardware consist of hierarchical layers of abstraction, each hiding the details of the layers below it from the layer above.

Definition 1.2 (Instruction Set Architecture). The *Instruction Set Architecture* (ISA) is the layer that abstracts the hardware/software interface. It allows many implementations of varying cost and performance to run the same software.



1.4 A Brief History

- **"Big Iron" computers** (e.g. ENIAC 1945, IBM Mark 1 1944): used vacuum tubes, electric relays and bulk magnetic storage devices. No microprocessors, no (stored-program) memory.
- **Von Neumann**: invented EDSAC (1949), the first *stored-program computer* – uses memory to hold both instructions and data. *We are still using this same basic design today.*
- **The processor chip**: the microprocessor evolution – Intel 4004 (1971) → 8086 (1978) → 80386 → 80486 → Pentium → Pentium II/III → ⋯ → multi-core Core i7, and today's Arrow Lake / Panther Lake (2026).

1.4.1 Selected microprocessor milestones (die data)

Chip	Year	Transistors	Area	Notes
Intel 4004	1970	2,250	12 mm^2	first microprocessor, 108 KHz
Intel 8086	1979	29,000	33 mm^2	5 MHz; basis of IA32 PC
Intel 80486	1989	1,200,000	81 mm^2	25 MHz; 1st pipelined IA32
Pentium	1993	3,100,000	296 mm^2	60 MHz; 1st superscalar IA32
Pentium III	1999	9,500,000	125 mm^2	450 MHz

1.4.2 Intel x86 evolution: milestones

Name	Date	Transistors	MHz	Notes
8086	1978	29K	5–10	First 16-bit Intel processor; basis for IBM PC & DOS; 1MB address space
386	1985	275K	16–33	First 32-bit Intel processor (IA32); flat addressing, can run Unix
Pentium 4E	2004	125M	2800–3800	First 64-bit Intel x86 processor (x86-64)
Core 2	2006	291M	1060–3333	First multi-core Intel processor
Core i7	2008	731M	1600–4400	Four cores

1.4.3 Process technology

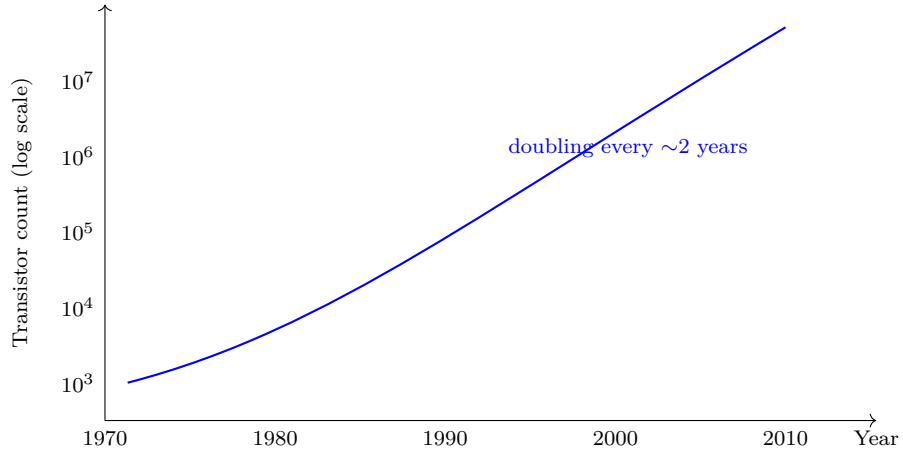
Generation	Process node
1st Pentium Pro (1995)	600 nm
1st Pentium III (1999)	250 nm
1st Pentium 4 (2000)	180 nm
1st Core 2 Duo (2006)	65 nm
Nehalem (2008)	45 nm
Sandy Bridge (2011)	32 nm
Ivy/Haswell/Broadwell/Skylake/Kaby/Coffee Lake (2012–2017)	22–14 nm
Ice/Tiger/Alder/Raptor Lake (2021–2023)	10 nm
Meteor / Lunar Lake (2024)	5–7 nm
Arrow Lake (2026)	3 nm
Panther Lake (2026)	∼2 nm (Intel 18A)

Process technology dimension = width of narrowest wires (10 nm ≈ 100 atoms wide).

1.4.4 Moore's Law

Definition 1.3 (Moore's Law). The number of transistors on a cost-effective integrated circuit doubles every 18–24 months. (Gordon Moore, "Cramming More Components onto Integrated Circuits," *Electronics*, 1965.)

Transistor count has grown from a few thousand (1970s) to over 100 billion transistors/chip today (e.g. Apple M2 Ultra, ∼134 billion).



Remark. The plot above is a schematic reconstruction of the trend, not the original dataset from the slides.

1.4.5 The end of the uniprocessor era

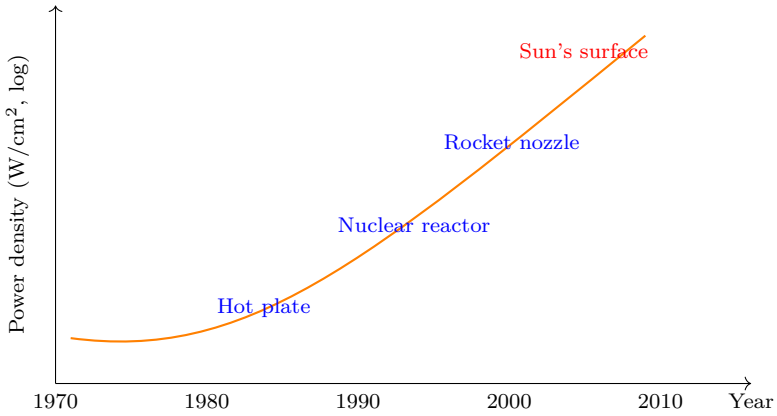
- "…today's processors …are nearing an impasse as technologies approach the speed of light…" – David Mitchell, *The Transputer: The Time Is Now* (1989).
- Custom multiprocessors strove to lead uniprocessors ⇒ procrastination rewarded: 2× sequential performance / 1.5 years (historically).
- "We are dedicating all of our future product development to multicore designs …This is a sea change in computing" – Paul Otellini, President, Intel (2004).
- All major microprocessor makers (AMD, Intel, IBM, Sun, Apple) switched to multicore ⇒ procrastination *penalized*: 2× sequential performance / 5(?) years. Biggest programming challenge: going from 1 to 2 CPUs.

Crossroads – uniprocessor performance growth (from Hennessy & Patterson):

Era	Annual growth
VAX (1978–1986)	∼25%/year
RISC + x86 (1986–2002)	∼52%/year
RISC + x86 (2002–present)	much slower (single-digit %/year)

1.4.6 Limiting force: power density

Chip power density has grown from the level of a *hot plate* (1980s), past a *nuclear reactor* (1990s), toward a *rocket nozzle* and approaching the *Sun's surface* by extrapolation – motivating a shift away from simply raising clock frequency.



1.4.7 Conventional wisdom, revisited

Old conventional wisdom	New conventional wisdom
Power is free, transistors expensive	**"Power wall"**: power expensive, transistors free (can put more on a chip than can afford to turn on)
Increase ILP via compilers, out-of-order, speculation, VLIW	**"ILP wall"**: law of diminishing returns on more hardware for ILP
Multiplies are slow, memory access is fast	**"Memory wall"**: memory slow, multiplies fast (∼200 clock cycles to DRAM vs. 4 clocks for a multiply)
Uniprocessor performance 2×/1.5 yrs	Power Wall + ILP Wall + Memory Wall = **Brick Wall**: uniprocessor performance now 2×/5(?) yrs

Note. Power Wall + ILP Wall + Memory Wall ⇒ a sea change in chip design: multiple "cores" (roughly 2× processors/chip every ∼2 years). It is more power-efficient to use many simpler processors than a few complex ones.

1.4.8 Sea change in chip design – an illustration

- Intel 4004 (1971): 4-bit, 2312 transistors, 0.4 MHz, 10 μm PMOS, 11 mm^2.
- RISC II (1983): 32-bit, 5-stage pipeline, 40,760 transistors, 3 MHz, 3 μm NMOS, 60 mm^2.
- A 125 mm^2, 65 nm CMOS chip could hold roughly 2312×(RISC II + FPU + Icache + Dcache) – *"the processor is the new transistor."*

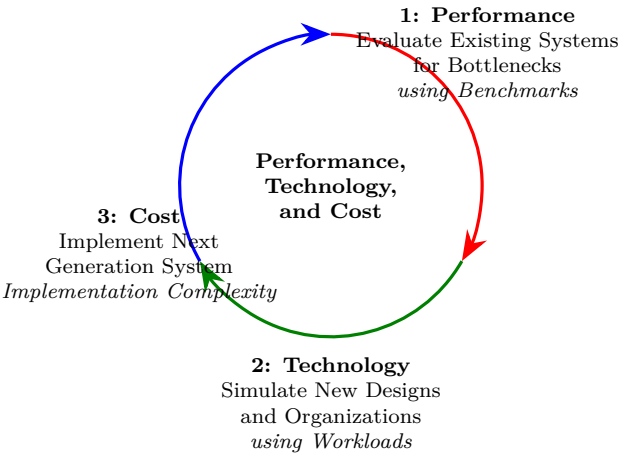
1.4.9 ManyCore chips

- Intel 80-core research chip (Feb 2007): 80 simple cores, two FP-engines/core, mesh network, 100M transistors, 65 nm.
- Intel Single-Chip Cloud Computer (Aug 2010): 24 "tiles" with two IA cores/tile, 24-router mesh network (256 GB/s bisection bandwidth), 4 integrated DDR3 controllers, HW support for message-passing.
- Open questions: how to program dozens/hundreds of cores? (e.g. dedicate cores to video/audio, one to the browser, dozens to virus scanning?) – something fundamentally new is needed in the software/architecture stack.

1.5 The Computer Design Cycle

A recurring three-part cycle drives computer design:

1. **Performance**: evaluate existing systems for bottlenecks using *benchmarks*.
2. **Technology**: simulate new designs and organizations, guided by *workloads* and *technology trends*.
3. **Cost**: implement the next-generation system, weighing *implementation complexity* against cost.



1.6 Performance and Cost

1.6.1 Performance metrics

Definition 1.4 (Latency and throughput)**.** **Time / latency** is the wall-clock or CPU elapsed time of a task. **Throughput** is the number of results produced per second.

Remark. Other measures (MIPS, MFLOPS, clock frequency, cache size) *do not make sense* on their own as performance measures.

1.6.2 Measuring tools

Benchmarks; hardware cost/delay/area/power consumption; simulation (at ISA, RT, gate, or circuit levels); queuing theory; fundamental laws/principles.

1.6.3 Price vs. cost

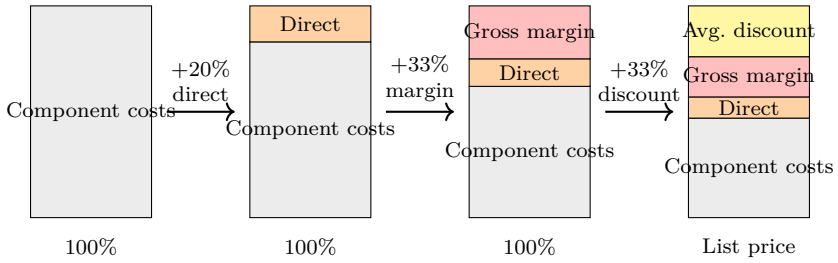
Definition 1.5 (Cost and price)**.** The **cost** is the total amount spent to produce a product. The **price** is the amount for which a finished good is sold.

The cost passes through several stages before becoming price; a small change in cost can have a large impact on price.

Manufacturing cost breakdown.

- **Component cost**: cost at which components (CPU, DRAM, disk, case, …) are available to the designer – typically 40–50% of list price.

- **Direct cost** (recurring): labor, purchasing, scrap, warranty – typically 4–16% of list price.
- **Gross margin** (non-recurring): R&D, marketing, sales, equipment, rental, maintenance, financing, pre-tax profit, taxes.
- **Average discount**: volume discounts and/or retailer markup, typically 15–35% of list price.



Component (% of list price)	Mini	W/S	PC
Component costs	∼20%	∼27%	∼55%
Direct costs	∼8%	∼8%	∼5%
Gross margin	∼47%	∼32%	∼20%
Average discount	∼25%	∼33%	∼20%

List price includes an average discount of 15–35% (volume discounts and/or retailer markup).

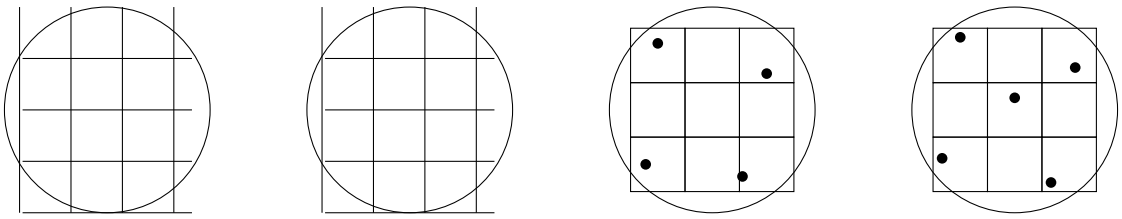
1.6.4 Cost-effective IC design

Definition 1.6 (Yield, volume, feature size)**.** **Yield** is the percentage of manufactured components surviving testing. **Volume** is the number of units manufactured; higher volume decreases the list price and improves purchasing efficiency. **Feature size** is the minimum size of a transistor or wire in either x or y.

Reduction in feature size from 10 μm (1971) to 0.045 μm (2008) has produced a quadratic rise in transistor count, a linear increase in performance, and a jump from 4-bit to 64-bit microprocessors.

1.6.5 Cost of integrated circuits

Manufacturing stages: wafer growth and testing → chopping the wafer into *dies* → packaging dies into *chips* → testing a chip. Dies near the round wafer periphery are wasted ("waist" area) since only whole squares fit.



Large die, poor fit | Better fit | Small die, few defects | Small die, more defects

Definition 1.7 (Die)**.** A *die* is the square area of the wafer containing the integrated circuit. Its cost is determined by the cost of a wafer, the number of dies that fit on the wafer, and

the fraction of dies that work (the *yield*).

Proposition 1.8 (Cost of an integrated circuit).

$$\text{Cost of IC} = \frac{\text{die cost} + \text{die testing cost} + \text{packaging cost} + \text{final testing cost}}{\text{final test yield}}$$

$$\text{Die cost} = \frac{\text{Cost of wafer}}{\text{dies per wafer} \times \text{die yield}}$$

$$\text{Dies per wafer} = \frac{\pi\,(\text{wafer diameter}/2)^2}{\text{die area}} - \frac{\pi\,(\text{wafer diameter})}{\sqrt{2 \times \text{die area}}}$$

Example (Dies per wafer). For a die of 0.7 cm on a side, find the number of dies per wafer of 30 cm diameter:

$$\frac{\pi(30/2)^2}{0.49} - \frac{\pi(30)}{\sqrt{2 \times 0.49}} = 1347 \text{ dies}$$

1.6.6 Calculating die yield

Proposition 1.9 (Die yield). Die yield is the fraction of good dies on a wafer. It depends on defect density D (defects per unit area) and a process-complexity parameter α (number of masking levels); a good estimate for CMOS is $\alpha = 4.0$:

$$\text{Die Yield} = \text{Wafer Yield} \times \left(1 + \frac{(\text{Defects/Unit Area}) \times \text{Die Area}}{\alpha}\right)^{-\alpha}$$

Example (Die yield). The yield of a die 0.7 cm on a side, with defect density $0.6/\text{cm}^2$:

$$\left(1 + \frac{0.6 \times 0.49}{4.0}\right)^{-4} = 0.75$$

1.6.7 Real-world example (1993 chip costs)

Chip	Layers	μm	Wafer \$	Def/cm^2	mm^2	Dies/wf	Yield	Die \$
386DX	2	0.90	\$900	1.0	43	360	71%	\$4
486DX2	3	0.80	\$1200	1.0	81	181	54%	\$12
PowerPC 601	4	0.80	\$1700	1.3	121	115	28%	\$53
HP PA 7100	3	0.80	\$1300	1.0	196	66	27%	\$73
DEC Alpha	3	0.70	\$1500	1.2	234	53	19%	\$149
Super SPARC	3	0.70	\$1700	1.6	256	48	13%	\$272
Pentium	3	0.80	\$1500	1.5	296	40	9%	\$417

From "Estimating IC Manufacturing Costs," Linley Gwennap, Microprocessor Report, Aug. 2, 1993.

1.6.8 Other costs

$$\text{Die Test Cost} = \frac{\text{Test Cost} \times \text{Average Test Time}}{\text{Die Yield}}$$

Packaging cost depends on pin count, heat dissipation, appearance, etc. Combining die cost

with test-and-assembly and packaging gives the total chip cost – e.g. Pentium (1993): \$417 (die) + \$56 (package + test) = \$473 total.

1.6.9 Volume vs. cost

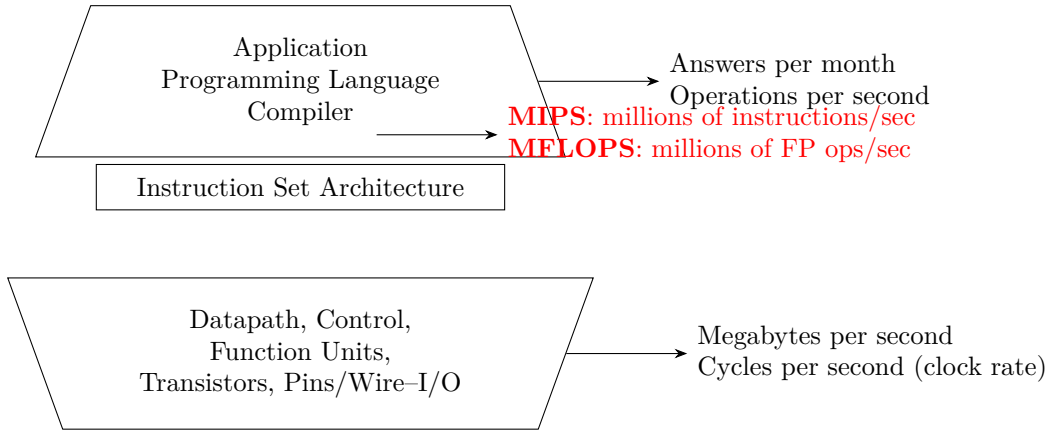
Note. Rule of thumb (learning-curve effect): "When volume doubles, costs reduce 10%." (*A DEC View of Computer Engineering*, C. G. Bell, J. C. Mudge, J. E. McNamara, 1978.)

Example (PC vs. workstation volume). Comparing PC vs. workstation shipment volumes (1990–1997), the PC/WS volume ratio grew from ∼59 to ∼67 ($2^x = 65 \Rightarrow x \approx 6.0$), so PC costs fall to roughly $(0.9)^{6.0} = 0.53$ of the original – PC costs end up about 47% lower than workstation costs for the same functionality.

1.6.10 High margins on high-end machines

R&D targets a 10% return on investment: every \$1 of R&D must generate \$7–\$13 in sales. High-end machines need more R&D dollars but sell in much lower volumes, so they carry much higher margins.

1.7 Metrics of Performance



1.7.1 Time

Definition 1.10 (CPU time). **CPU time** is the time the CPU is computing, not including time waiting for I/O or running other programs. **User CPU time** is the CPU time spent in the program; **system CPU time** is the CPU time spent in the OS performing tasks requested by the program.

$$\text{CPU time} = \text{User CPU time} + \text{System CPU time}$$

Example (Unix `time` command). `90.7u 12.9s 2:39 65%` ⇒ user CPU 90.7 s, system CPU 12.9 s, elapsed (response) time 2:39 = 159 s; $(90.7 + 12.9)/159 \times 100 = 65\%$ of elapsed time is CPU time – the other 35% is spent in I/O or running other programs.

1.7.2 Two notions of "performance"

Example (Boeing 747 vs. Concorde).

Plane	DC to Paris	Speed	Passengers	Throughput (pmph)
Boeing 747	6.5 hours	610 mph	470	286,700
BAD/Sud Concorde	3 hours	1350 mph	132	178,200

- **Time to do the task** (execution time, response time, latency): Concorde is 1350/610 = 2.2× faster (= 6.5/3 hours).
- **Tasks per unit time** (throughput, bandwidth): Boeing is 286,700/178,200 = 1.6× (60%) higher throughput; Concorde is only 0.62× the Boeing's throughput.

Remark. Response time and throughput often trade off against each other.

1.7.3 Price-performance design: train vs. plane

Example (Train vs. plane).

Vehicle	Time lab-to-lab	Passengers/trip	Time to complete job	Exec. time/person
Train	4.0 hours	2400	4.0 hours	6.0 sec
Plane	45 min	300	45×8=6.0 hr	9.0 sec

The plane is over 5× faster individually but needs 50% more time to finish the whole job (2400 passengers) – i.e. lower throughput; the train's per-person time and cost are both lower, so its *cost-performance* is about 15× better than the plane's (illustrative example).

1.8 Comparing Performance and Speedup

1.8.1 Execution-time-based performance

Definition 1.11 (Performance and speedup). For a given program,

$$\text{Performance}_A = \frac{1}{\text{Execution Time}_A}.$$

"Machine A is n times faster than machine B" (running the same program) means

$$\text{Speedup} = n = \frac{\text{Performance}_A}{\text{Performance}_B} = \frac{\text{Execution Time}_B}{\text{Execution Time}_A}.$$

Example. $\text{Exec}_A = 1$ s, $\text{Exec}_B = 10$ s $\Rightarrow$ Speedup $= 10/1 = 10$: machine A is 10 times faster.

1.8.2 The CPU equation

Theorem 1.12 (The CPU equation). Let a program execute an *instruction count* I (instructions/program), with an average of CPI *cycles per instruction*, on a CPU with *clock cycle time* $C = 1/\text{clock rate}$ (seconds/cycle). Then

$$\text{CPU time} = \frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Cycle}}, \qquad \text{i.e.} \qquad T = I \times \text{CPI} \times C.$$

Example (CPU time). I = 10,000,000 instructions, average CPI = 2.5, clock rate = 200 MHz ($C = 5 \times 10^{-9}$ s):

$$\text{CPU time} = 10{,}000{,}000 \times 2.5 \times 5 \times 10^{-9} = 0.125 \text{ seconds}$$

1.8.3 Factors affecting CPU performance

	Instruction Count I	CPI	Clock Cycle C
Program	X	X	
Compiler	X	X	
Instruction Set Architecture (ISA)	X	X	
Organization (CPU design)		X	X
Technology (VLSI)			X

1.8.4 Performance-comparison example: changing compiler and CPU

Example (New compiler and faster CPU). Baseline: I = 10,000,000, CPI= 2.5, clock rate = 200 MHz. With a new compiler (I' = 9,500,000, CPI$'$ = 3.0) and a faster CPU (clock rate = 300 MHz):

$$\text{Speedup} = \frac{I \times CPI \times C}{I' \times CPI' \times C'} = \frac{10{,}000{,}000 \times 2.5 \times 5 \times 10^{-9}}{9{,}500{,}000 \times 3.0 \times 3.33 \times 10^{-9}} = \frac{0.125}{0.095} = 1.32$$

i.e. 32% faster after the changes.

1.8.5 Instruction types & CPI

Proposition 1.13 (CPI from instruction classes). For a program with n instruction classes, C_i = count of instructions of type i, CPI_i = cycles per instruction of type i:

$$\text{CPU clock cycles} = \sum_{i=1}^{n} (CPI_i \times C_i) \qquad I = \sum_i C_i \qquad CPI = \frac{\text{CPU clock cycles}}{I}$$

Example (Two code sequences). 3 instruction classes with $CPI_A = 1$, $CPI_B = 2$, $CPI_C = 3$. Two code sequences:

Sequence	C_A	C_B	C_C
1	2	1	2
2	4	1	1

Sequence 1: cycles = $2(1) + 1(2) + 2(3) = 10$, $I = 5$, $CPI = 10/5 = 2$. Sequence 2: cycles = $4(1) + 1(2) + 1(3) = 9$, $I = 6$, $CPI = 9/6 = 1.5$.

1.8.6 Instruction frequency & CPI

Proposition 1.14 (CPI from instruction frequencies). Let $F_i = C_i/I$ be the frequency (frac-

tion) of instruction type i. Then

$$CPI = \sum_{i=1}^{n} (CPI_i \times F_i) \qquad \%\text{ of exec. time on type } i = \frac{CPI_i \times F_i}{CPI}$$

Example (RISC typical instruction mix).

Op	F_i	CPI_i	$CPI_i \times F_i$
ALU	50%	1	0.50
Load	20%	5	1.00
Store	10%	3	0.30
Branch	20%	2	0.40
Total	100%	$CPI = 2.20$	

1.8.7 Performance terminology

Definition 1.15 ("n% faster"). "X is n% faster than Y" means

$$\frac{\text{ExTime}(Y)}{\text{ExTime}(X)} = \frac{\text{Performance}(X)}{\text{Performance}(Y)} = 1 + \frac{n}{100}, \quad \text{i.e.}$$
$$n = \frac{100\,(\text{ExTime}(Y) - \text{ExTime}(X))}{\text{ExTime}(X)}.$$

Example. Y takes 15 s, X takes 10 s: $n = 100(15-10)/10 = 50\%$ faster.

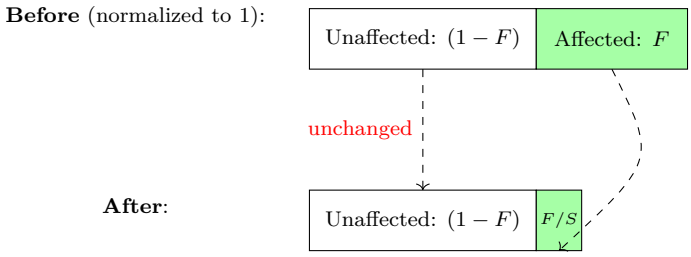
1.9 Amdahl's Law

Theorem 1.16 (Amdahl's Law). The performance improvement to be gained from using a faster mode of execution is limited by the fraction of time the faster mode can actually be used. If an enhancement E accelerates a fraction F ($\text{Fraction}_{enhanced}$) of a task by a factor S ($\text{Speedup}_{enhanced}$), and the remainder of the task is unaffected, then

$$\text{ExTime}_{new} = \text{ExTime}_{old} \times \left[(1-F) + \frac{F}{S}\right] \qquad \text{Speedup}_{overall} = \frac{1}{(1-F) + \frac{F}{S}}.$$

Proof. Normalize $\text{ExTime}_{old} = 1 = (1-F) + F$. The unaffected part still takes $1-F$; the affected part now takes F/S. So $\text{ExTime}_{new} = (1-F) + F/S$, and Speedup $= \text{ExTime}_{old}/\text{ExTime}_{new}$. □

Explanation. Even an infinitely fast enhancement ($S \to \infty$) gives at most Speedup $\to 1/(1-F)$: the part you did not improve becomes the bottleneck. F must be a fraction of *execution time*, not of instruction count.



1.9.1 Worked examples

Example (Floating point, single enhancement). FP instructions improved to run 2× faster; only 10% of instructions are FP:

$$\text{ExTime}_{new} = \text{ExTime}_{old} \times \left((1-0.1) + \frac{0.1}{2}\right) = 0.95\,\text{ExTime}_{old}$$

$$\text{Speedup} = \frac{1}{0.95} = 1.053$$

Example (RISC load-CPI enhancement). Using the RISC instruction mix above, a design enhancement improves the CPI of `Load` from 5 to 2. `Load` instructions are 20% of the instruction count, but the fraction of *execution time* they affect is what matters for Amdahl's Law: from the earlier % Time table, Load accounts for $F = 45\% = 0.45$ of execution time. $S = 5/2 = 2.5$:

$$\text{Speedup}(E) = \frac{1}{(1-0.45) + 0.45/2.5} = \frac{1}{0.55 + 0.18} = 1.37$$

Cross-check using the CPU equation directly: Old $CPI = 2.2$; New $CPI = 0.5(1) + 0.2(2) + 0.1(3) + 0.2(2) = 1.6$; Speedup = old CPI/new $CPI = 2.2/1.6 = 1.37$ – matches.

Example (How much must multiplication speed up?). A program runs in 100 s on a machine where multiply operations take 80 s of that time. By how much must multiply be sped up to make the whole program:

- **4× faster?** Need $\text{ExTime}_{new} = 25$s: $25 = (100-80) + 80/n \Rightarrow n = 80/5 = 16$; multiply must be 16× faster.
- **5× faster?** Need $\text{ExTime}_{new} = 20$s: $20 = 20 + 80/n \Rightarrow 0 = 80/n$ – **impossible**: no amount of multiply speed-up can achieve a 5× overall speedup, since the unaffected 20 s alone is already the target time.

1.9.2 Extending Amdahl's Law to multiple enhancements

Corollary 1.17 (Multiple enhancements). If enhancements E_i each accelerate a (disjoint) fraction F_i of the original execution time by factor S_i, then

$$\text{Speedup} = \frac{1}{(1 - \sum_i F_i) + \sum_i \frac{F_i}{S_i}}.$$

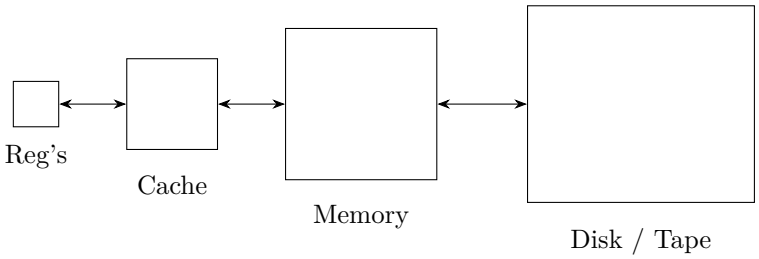
All fractions F_i refer to the *original*, un-enhanced execution time.

Example (Three enhancements). $S_1 = 10, F_1 = 20\%$; $S_2 = 15, F_2 = 15\%$; $S_3 = 30, F_3 = 10\%$, each affecting a disjoint portion of the code, all applied simultaneously:

$$\begin{aligned}\text{Speedup} &= \frac{1}{(1 - .2 - .15 - .1) + .2/10 + .15/15 + .1/30} \\ &= \frac{1}{.55 + .02 + .01 + .0033} = \frac{1}{.5833} = 1.71\end{aligned}$$

1.9.3 Corollary: make the common case fast

- All instructions require an instruction fetch; only a fraction require a data fetch/store ⇒ optimize instruction access over data access.
- Programs exhibit *locality* (e.g. 90% of time in 10% of code): spatial locality and temporal locality.
- Access to small memories is faster ⇒ provide a *storage hierarchy* so the most frequent accesses go to the smallest, closest memories.



1.10 Marketing Metrics: MIPS and MFLOPS

1.10.1 MIPS

Definition 1.18 (MIPS rating).

$$\text{MIPS} = \frac{\text{Instruction Count}}{\text{Execution Time} \times 10^6} = \frac{\text{Clock Rate}}{CPI \times 10^6}$$

MIPS is a rate of operations per unit time; naive reasoning says a bigger MIPS rating means a faster machine.

Wrong! Three significant problems with using MIPS:

1. **Instruction-set dependent** – different machines/ISAs are not comparable.
2. **Varies between programs** on the *same* computer.
3. **Can vary inversely with performance!**

Example (Higher MIPS, slower program). Same machine, instruction classes A (CPI 1), B (CPI 2), C (CPI 3), clock = 100 MHz. Two compilers:

Compiler	C_A (M)	C_B (M)	C_C (M)
1	5	1	1
2	10	1	1

$$CPI_1 = \frac{5(1)+1(2)+1(3)}{5+1+1} = \frac{10}{7} = 1.43 \qquad MIPS_1 = \frac{100}{1.43} = 69.9$$

$$CPI_2 = \frac{10(1)+1(2)+1(3)}{10+1+1} = \frac{15}{12} = 1.25 \qquad MIPS_2 = \frac{100}{1.25} = 80.0$$

Compiler 2 has the higher MIPS rating – so is it faster? Compare CPU *time* instead:

$$\text{CPU time}_1 = \frac{7 \times 10^6 \times 1.43}{100 \times 10^6} = 0.10 \text{ s} \qquad \text{CPU time}_2 = \frac{12 \times 10^6 \times 1.25}{100 \times 10^6} = 0.15 \text{ s}$$

Compiler 1's program is actually faster, despite its lower MIPS rating! – because it executes fewer instructions overall.

Note. MIPS is valid for comparison only when (1) the same program is used, (2) the same ISA is used, and (3) the same compiler is used – i.e. the compared programs are identical down to the instruction count.

1.10.2 MFLOPS

Definition 1.19 (MFLOPS rating).

$$\text{MFLOPS} = \frac{\text{Number of floating-point operations}}{\text{Execution time} \times 10^6}$$

A floating-point operation is an add/sub/mul/div on single- or double-precision values. MFLOPS is generally a better cross-machine comparison than MIPS (applies even across different ISAs), but it is still **program-dependent** (different programs have different fractions of FP ops; a compiler benchmark, with no FP ops, yields MFLOPS = 0) and dependent on the *mix* of FP operation types present. A *Peak MFLOPS* rating – obtained from a program of only the simplest FP instructions – does not represent real FP programs.

1.11 Benchmarks

1.11.1 CPU benchmark suites

Definition 1.20 (Benchmark). A *benchmark* is a program specifically chosen to measure performance.

Five levels of programs, in decreasing order of realism/accuracy:

1. Real applications
2. Modified (scripted) applications
3. Kernels (extracted hot loops)
4. Toy benchmarks
5. Synthetic benchmarks

1.11.2 SPEC (System Performance Evaluation Cooperative)

- **SPECCPU**: popular desktop CPU-only benchmark suite, split into integer and floating-point programs.
- **1989 (1st round)**: 10 programs → a single SPECmark number.

- **1992 (2nd round)**: SPECint92 (6 integer) and SPECfp92 (14 FP) programs.
- **1995 (3rd round)**: SPECint95 (8 integer), SPECfp95 (10 FP); "benchmarks useful for 3 years"; a single flag setting for all programs (SPECint_base95, SPECfp_base95).
- **SPECint2000/SPECfp2000**, and **SPECCPU2006**.
- **SPECSFS** (NFS file server) and **SPECWeb** (web server) added as server benchmarks.

1.12 Summarizing Performance

1.12.1 Arithmetic mean and weighted arithmetic mean

$$\text{Arithmetic mean} = \frac{1}{n}\sum_{i=1}^{n} T_i \qquad \text{or weighted:} \qquad \sum_i W_i\, T_i$$

Tracks (weighted) *total* execution time, but gives more weight to longer-running programs, and different weight choices can make different machines "look better."

Example (Which machine is better?). Two programs P1, P2 on three computers:

	Computer A	Computer B	Computer C
P1 (sec)	1	10	20
P2 (sec)	1000	100	20
Total time	1001	110	40

Which machine is "better"? With three different weighting schemes for (P1,P2):

Weights (P1/P2)	Computer A	Computer B	Computer C
.5 / .5	500.5	55.0	20
.909 / .091	91.82	18.8	20
.999 / .001	2	10.09	20

Each weighting scheme picks a different "winner" – illustrating why the choice of weights is itself contentious.

1.12.2 Normalized execution time and the geometric mean

Definition 1.21 (Normalized execution time). Pick a reference machine R. The normalized execution time of program A on machine X is (time to run A on X) / (time to run A on R). Summarize with the **geometric mean**

$$\sqrt[n]{\prod_{i=1}^{n} \text{Normalized execution time on } i.}$$

Remark. The arithmetic mean of normalized times depends on the choice of reference machine; the geometric mean is *consistent regardless of the reference machine*, but it is *not* a good metric for total execution time.

1.12.3 Summary of means

Mean	Formula	Property
Arithmetic	$\frac{1}{n}\sum_i T_i$	Can be weighted; represents total execution time
Harmonic	$\frac{n}{\sum_i 1/R_i}$	Used for rates (e.g. throughput)
Geometric	$\left(\prod_i \frac{T_i}{T_{r_i}}\right)^{1/n} = \exp\left(\frac{1}{n}\sum_i \log\frac{T_i}{T_{r_i}}\right)$	Consistent, independent of reference machine

1.12.4 Performance evaluation, in practice

Sales are, in part, a function of performance relative to the competition, and companies invest heavily in improving the product as reported by performance summaries. Good products result from having *good benchmarks* and *good ways to summarize performance*. If benchmarks or summaries are inadequate, vendors must choose between improving the product for real programs vs. improving it to score better on the benchmark – and, cynically, "sales almost always wins." In the end, **execution time remains the real measure of computer performance**.

1.13 Summary

- Chip density continues to increase ($\sim 2\times$ every 2 years); clock speed has largely stopped scaling, so the number of processors/cores per chip increases instead.
- Always keep concrete metrics and benchmarks in mind to justify comparisons between systems.
- **Amdahl's Law**:

$$\text{Speedup}_{overall} = \frac{\text{ExTime}_{old}}{\text{ExTime}_{new}} = \frac{1}{(1 - \text{Fraction}_{enhanced}) + \frac{\text{Fraction}_{enhanced}}{\text{Speedup}_{enhanced}}}$$

- **The CPU equation**:

$$\text{CPU time} = \frac{\text{Seconds}}{\text{Program}} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Cycle}}$$

- **Execution time is the only reliable measure of computer performance**; die cost grows roughly with die area to a power greater than 1 (quadratically-ish, via the yield model).
- "For better or worse, benchmarks shape a field." Good products need good benchmarks *and* good ways to summarize performance.

2 Lecture 2: Review of Instruction Set Architecture

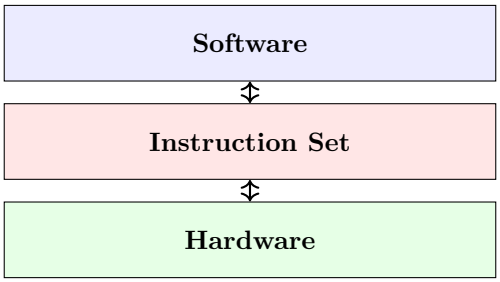
September 20, 2026

2.1 Overview

This lecture reviews the instruction set—the interface between software and hardware. We compare ISA styles (stack, accumulator, memory-memory, register-memory, load-store), the main design dimensions of an instruction set (operations, addressing modes, operand types, encoding, registers), CISC vs. RISC, and then study MIPS in detail: its instruction formats, register conventions, procedure calls, and assembler usage.

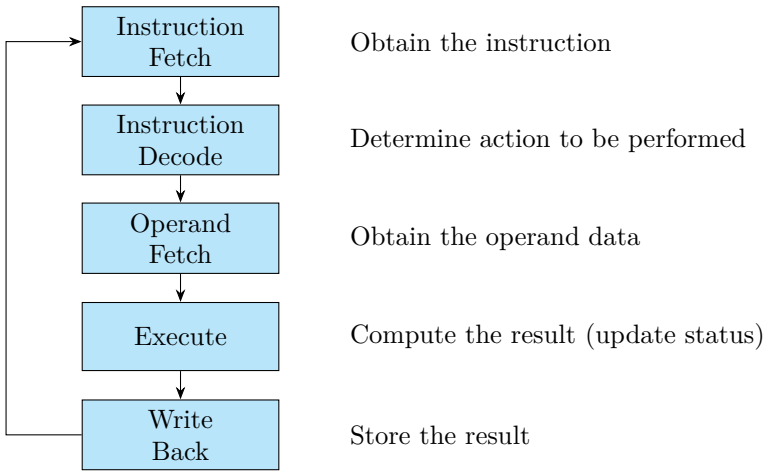
2.2 The Instruction Set

Definition 2.1 (Instruction, instruction set)**.** A computer can only understand binary values; its operation is defined by predefined binary values called **instructions**. The **instruction set** is the set of all instructions a processor can perform.



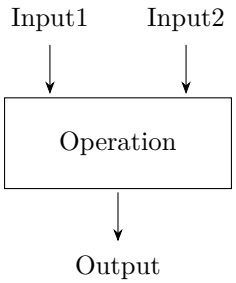
Note. The instruction set is the **interface** between software and hardware.

2.2.1 Instruction execution cycle



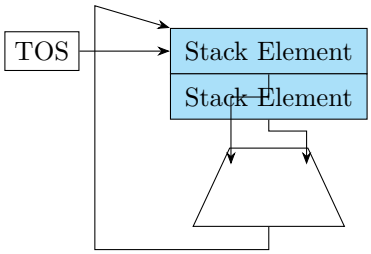
2.3 ISA Styles

A generic 2-operand operation:



Definition 2.2 (ISA style). ISA styles differ in *where* the operands of an operation live: **Stack**, **Accumulator**, **Register-Memory / Memory-Memory**, and **Register-Register** (a.k.a. *load-store*).

2.3.1 Stack



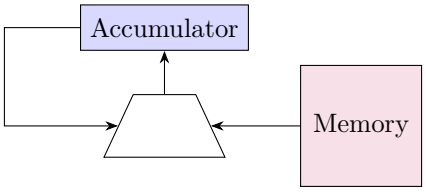
The operands are on top of the stack; the result is pushed back onto the stack.

Example ($C = A + B$).

```
PUSH A
PUSH B
ADD
POP C     # C = A + B
```

Remark. (+) Good code density, simple hardware. (−) Low parallelism, stack is a bottleneck.

2.3.2 Accumulator



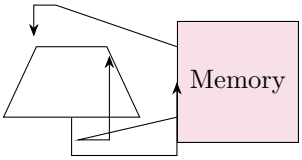
One accumulator register is used in all operations.

Example ($C = A + B$).

```
LOAD A    ; put A in Accumulator (AC)
ADD B     ; AC <- AC + B
STORE C   ; C <- AC
```

Remark. (+) Easy to write a compiler, few instructions. (–) Very high memory traffic, variable CPI.

2.3.3 Memory-Memory



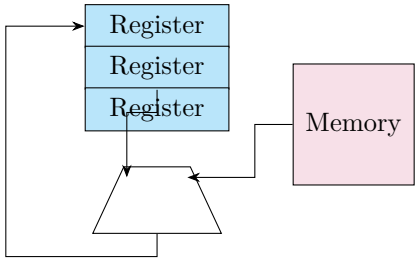
All operands are located in memory.

Example ($C = A + B$).

```
ADD C, A, B    ; C <- A + B   (all three in memory)
```

Remark. (+) Simple to design and understand. (–) Accumulator-like bottleneck, heavy memory access.

2.3.4 Register-Memory

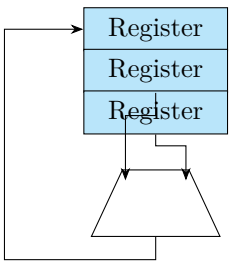


Inputs/outputs can be either a register or memory.

Example ($C = A + B$).

```
LOAD  R1, A
ADD   R3, R1, B    ; R3 <- R1 + mem[B]
STORE R3, C
```

2.3.5 Register-Register (Load-Store)



All operations are on registers; dedicated `Load`/`Store` instructions access memory.

Example ($C = A + B$).

```
LOAD  R1, A
LOAD  R2, B
ADD   R3, R1, R2
STORE R3, C
```

2.3.6 Historical examples

Machine	#GPRs	Architecture style	Year
Motorola 6800	2	Accumulator	1974
DEC VAX	16	Register-Memory / Memory-Memory	1977
Intel 8086	1	Extended Accumulator	1978
Motorola 68000	16	Register-Memory	1980
Intel 80386	32	Register-Memory	1985
Power PC	32	Load-Store	1992
Dec Alpha	32	Load-Store	1992

2.3.7 Other ISA styles

- **High-level-language architecture** (e.g. Burroughs B5000, 1960s): instructions map closely to HLL constructs. Fell out of favor due to a lack of effective compilers to exploit it.
- **Reduced Instruction Set (RISC) architecture**: simplify the hardware, the instruction set, and the instruction format; rely on the compiler to synthesize complex operations from simple ones. (Covered in detail below.)

2.4 Instruction Set Design

The design of an instruction set is critical to the operation of a computer system, and includes many aspects:

- Operation repertoire
- Addressing modes
- Data types
- Instruction format
- Registers

2.4.1 Simple instruction format



Definition 2.3 (Instruction fields).

- **Operation Code**: the operation to be performed by the processor.
- **Source Operand Reference**: input(s) of the operation — one or more source operands can be involved.

- **Result Operand Reference**: result of the operation.

2.4.2 Instruction types

Can be classified into 4 types:

- **Data processing**: Arithmetic, Logic (e.g. `ADD`, `SUB`, `AND`, `OR`, …)
- **Data storage**: Move data from/to memory (e.g. `LD`, `ST`)
- **Data movement**: Register and register/IO (e.g. `MOV`)
- **Control**: Test and branch (e.g. `JMP`, `CMP`)

Explanation. *"There must certainly be instructions for performing the fundamental arithmetic operations"* — Burkes, Goldstine and Von Neumann, 1947. How many programs have an `IF` statement? → branch instructions. How many have a `Call` statement? → call/return instructions. How many programs access memory? …and so on: this is how an ISA's operation repertoire is decided.

2.4.3 Operator types

Operator type	Example
Arithmetic & Logical	Integer arithmetic and logical operations: add, and, subtract …
Data transfer	Loads-stores (move instructions on machines with memory addressing)
Control	Branch, jump, procedure call and return, trap
System	Operating system call, virtual memory management instructions
Floating point	Floating point instructions: add, multiply
Decimal	Decimal add, decimal multiply, decimal-to-character conversion
String	String move, string compare, string search
Graphic	Pixel operations, compression/decompression operations

Remark. Arithmetic, logical, data transfer and control are almost standard categories for all machines. System instructions are required for multi-programming environments (support varies). Others can be primitives (e.g. decimal and string on IBM 360 and VAX), provided by a co-processor, or synthesized by the compiler.

2.5 Addressing Modes

Definition 2.4 (Addressing mode). The way the processor refers to the operands is called the **addressing mode**.

Addressing modes can be classified based on:

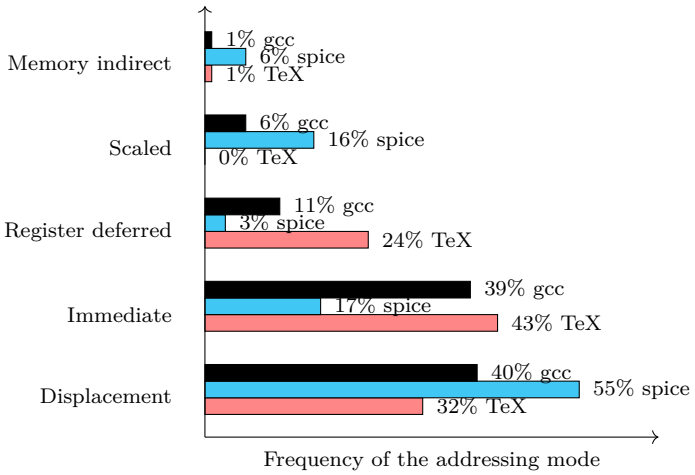
- **The source of data**: immediate, registers, memory.
- **The address calculation**: direct, indirect, indexed.

Mode	Description	Example
Immediate	The operand is put in the instruction.	`ADD R0,#10`
Register	The index of the register containing the operand is specified in the instruction.	`ADD R0,R1`
Direct	The address of the operand is put in the instruction.	`ADD R0,(100)`
Register Indirect	The address of the operand is put *in the register* specified in the instruction.	`ADD R0,(R1)`
Displacement	The address of the operand is Base register + Displacement.	`LD R1,100(R2)`
Indexed	The address of the operand is Base register + Index register.	`ADD R3,(R1+R2)`

Generic notation (pseudo-RTL) for the same set of modes:

Mode	Example
Immediate	`add r1 = r2 + 5`
Register	`add r1 = r2 + r3`
Direct	`load r1 = M[4000]`
Register Indirect	`add r1 = r2 + M[r2]`
Displacement	`load r1 = M[r2 + 4000]`
Indexed/Base	`add r1 = r3 + M[r2 + r3]`
Memory Indirect	`load r1 = M[M[r2]]`
PC Relative	`branch r1 < r3, 1000`
Scaled	`load r1 = M[100 + r3 + r4*d]`

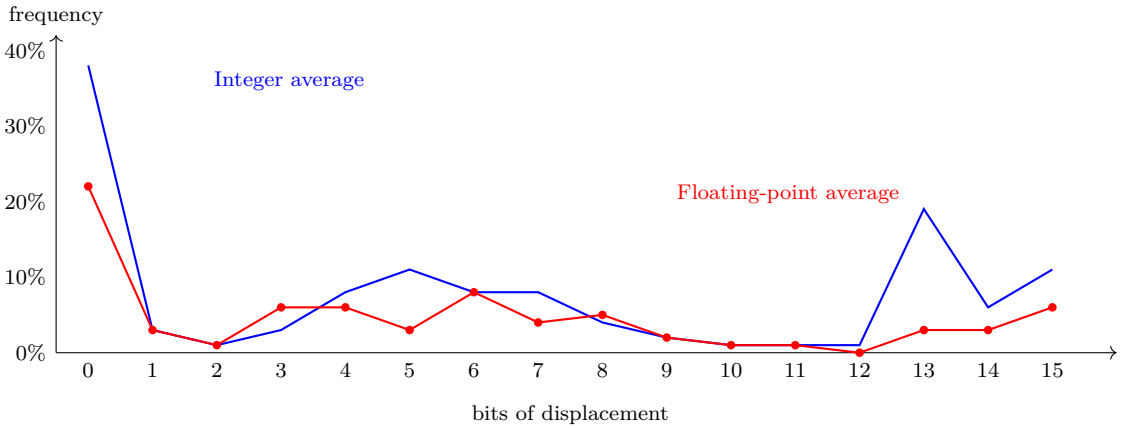
2.5.1 Addressing mode usage (SPEC89 on VAX)



Focus on immediate and displacement modes since they are used the most across TeX, spice, and gcc.

2.5.2 Displacement value size

Measured as number of bits needed; both integer-average and floating-point-average workloads show most displacement values fit in a small number of bits, with the bulk of accesses needing ≤ 15 bits and a long tail:



2.5.3 Design questions raised by addressing modes

- Is Memory-indirect addressing necessary?
- Is Scaled addressing necessary?
- Is Register addressing necessary?
- How long should a displacement value be?
- How long should an immediate value be?

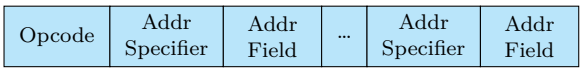
2.6 Operand Types and Sizes

- **Character**:
  - ASCII (8-bit): almost always used.
  - Unicode (16-bit): sometimes.
- **Integer**: 2's complement.
  - Short: 16 bit.
  - Long: 32 bit.
- **Floating point**:
  - Single precision: 32 bit.
  - Double precision: 64 bit.
- **Business**: Binary Coded Decimal (BCD) – accurately represents decimal fractions.
- **DSP**: Fixed point, block floating point.
- **Graphic**: RGBA or XYZW, using 8-bit, 16-bit, or single precision floating point components.

> **Remark.** Design questions: Should the CPU support all these operand types? Should it support very big-size operands? Is DSP data used frequently enough to add dedicated support? Is BCD used in most operations? How about RGBA?

2.7 Instruction Format

- Instructions must be encoded to binary values.
- This affects the size of the compiled program.
- Easy to decode ⇒ simple to implement.
- Should support as many registers and addressing modes as possible.



Variable instruction length (e.g. VAX, X86)



Fixed instruction length (e.g. ARM, MIPS, PowerPC)

Note. Hybrid: to gain high code density while keeping decode simple, use 2 types of fixed-length instructions (e.g. MIPS16, Thumb).

2.7.1 Register file

- Register is the fastest memory element.
- Registers cost much more (per bit) than main memory.
- Registers are flexible for the compiler to use.
- More registers need more bits to encode in each instruction.
- A register file with more locations can be slower to access.
- Open design question: how many locations in the register file is most effective?

2.8 CISC vs. RISC

2.8.1 Complex Instruction Set Computer (CISC)

Definition 2.5 (CISC). A *Complex Instruction Set Computer* emphasizes doing more with each instruction, so programs need fewer instructions (more compact code).

- Emphasizes doing more with each instruction ⇒ fewer instructions per program (more compact code).
- Motivated by the **high cost of memory** and hard-disk capacity when original CISC architectures were proposed:
  - M6800: 16K RAM = $500, 40M hard disk = $55,000.
  - M68000: 64K RAM = $200, 10M HD = $5,000.
- Original CISC architectures evolved with faster, more complex CPU designs, but backward instruction-set compatibility had to be maintained.
- Wide variety of addressing modes: 14 in MC68000, 25 in MC68020.
- A number of instruction modes for the location and number of operands: the VAX has 0- through 3-address instructions.
- Variable-length instruction encoding.

Example (Motorola 680X0)**.** 18 addressing modes (data/address register direct, immediate, absolute short/long, address register indirect with post-/pre-increment, displacement, indexed, memory indirect pre-/post-indexed, PC-relative variants, ...). GPR ISA (Register-Memory). Operand size ranges from 1 to 32 bits (1, 2, 4, 8, 10, or 16 bytes). Instructions are stored in 16-bit words; the smallest instruction is 2 bytes (one word); the longest is 5 words (10 bytes).

Example (Intel IA-32, x86 (80386))**.** 12 addressing modes (register, immediate, direct, base, base+displacement, index+displacement, scaled-index+displacement, based index, based scaled index, based index+displacement, based scaled index+displacement, relative). GPR ISA (Register-Memory). Operand sizes 8, 16, 32, 48, 64, or 80 bits; also supports string operations. The smallest instruction is one byte; the longest is 12 bytes. The first bytes generally contain the opcode, mode specifiers, and register fields; the remainder are for address displacement and immediate data.

2.8.2 Reduced Instruction Set Computer (RISC)

Definition 2.6 (RISC)**.** A *Reduced Instruction Set Computer* focuses on reducing the number and complexity of instructions, aiming for CPI ≤ 1.

- Focuses on reducing the number and complexity of instructions.
- Reduced CPI: goal of at least one instruction per clock cycle (CPI = 1 or less).
- Designed with pipelining in mind.
- Fixed-length instruction encoding.
- Only load and store instructions access memory.
- (Thus more instructions are executed than CISC.)
- Simplified addressing modes – usually limited to immediate, register indirect, register displacement, indexed.
- Delayed loads and branches.
- Instruction pre-fetch and speculative execution.
- Examples: MIPS, SPARC, PowerPC, Alpha.

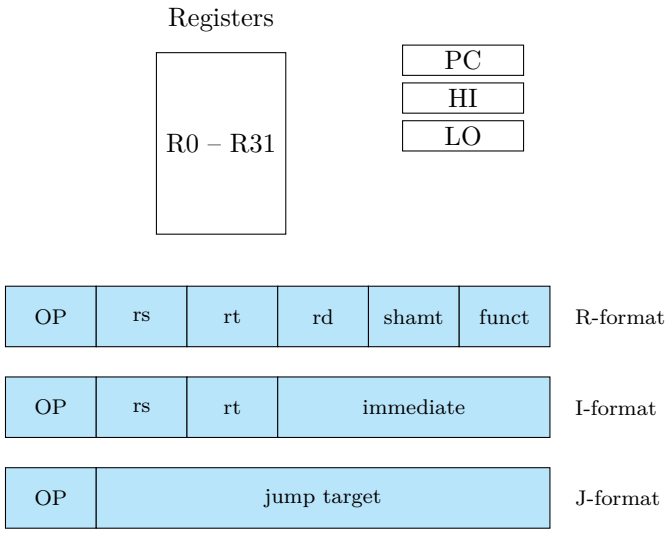
Example (HP Precision Architecture (HP PA-RISC))**.** 7 addressing modes: register, immediate, base with displacement, base with scaled index and displacement, predecrement, postincrement, PC-relative. Five operand sizes ranging in powers of two from 1 to 16 bytes. Instruction set has 12 different formats, all 32 bits in length.

Example (Alpha AXP)**.** 4 addressing modes: register direct, immediate, register indirect with displacement, PC-relative. Four operand sizes: 1, 2, 4, or 8 bytes. Instruction set has 7 different formats, all 32 bits in length.

2.9 Case Study: MIPS

2.9.1 Overview

- Instruction categories: Load/Store, Computational, Jump and Branch, Floating Point (coprocessor), Memory Management, Special.
- 3 instruction formats: all **32 bits wide**.



2.9.2 MIPS (RISC) design principles

MIPS (RISC) design principles

- **Simplicity favors regularity**: fixed-size instructions, small number of instruction formats, opcode always in the first 6 bits.
- **Smaller is faster**: limited instruction set, limited number of registers in the register file, limited number of addressing modes.
- **Make the common case fast**: arithmetic operands from the register file (load-store machine), allow instructions to contain immediate operands.
- **Good design demands good compromises**: same instruction length, single instruction format ⇒ compromise of exactly 3 instruction formats.

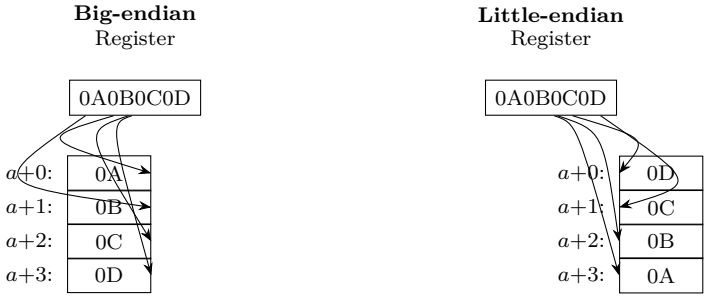
2.9.3 MIPS instruction-class distribution (SPEC2006)

Instruction Class	Integer	Floating Point
Arithmetic	16%	48%
Data transfer	35%	36%
Logical	12%	4%
Cond. Branch	34%	8%
Jump	2%	0%

2.9.4 MIPS register convention

Name	Register #	Usage	Preserve on call?
$zero	0	constant 0 (hardware)	n.a.
$at	1	reserved for assembler	n.a.
$v0–$v1	2–3	returned values	no
$a0–$a3	4–7	arguments	yes
$t0–$t7	8–15	temporaries	no
$s0–$s7	16–23	saved values	yes
$t8–$t9	24–25	temporaries	no
$k0–$k1	26–27	reserved for operating system	n.a.
$gp	28	global pointer	yes
$sp	29	stack pointer	yes
$fp	30	frame pointer	yes
$ra	31	return address (hardware)	yes

2.9.5 MIPS endianness



Definition 2.7 (Endianness).

- **Big-endian**: most-significant byte at the lowest address of a word.
- **Little-endian**: least-significant byte at the lowest address of a word.
- **MIPS is big-endian.**

2.9.6 R-Format instructions



Definition 2.8 (R-format fields).

- **Op**: opcode.
- **Rs**: first source register number.
- **Rt**: second source register number.
- **Rd**: destination register number.
- **Shamt**: shift amount (number of bit positions, left/right).
- **Funct**: extended opcode – the ALU function that encodes the datapath operation. Execution: `Rd <- Rs func Rt`.

For arithmetic and logical instructions the opcode is always `SPECIAL` (`000000`), and `Funct` indicates the specific operation.

Example (Encoding `ADD $t0, $s1, $s2`).

`ADD $t0, $s1, $s2`

Special	$s1	$s2	$t0	0	add
0	17	18	8	0	32

Binary: `000000 10001 10010 01000 00000 100000` = `0x02324020`.

2.9.7 Logical operations

`AND, OR, XOR, NOR` (bit-wise).

```
OR $t0, $t1, $t2    # $t0 = $t1 | $t2
```

Shift left / shift right: `Shamt` indicates the number of bits to shift.

Example (Shift left logical).

`SLL $t2, $s0, 8`

Encoding: `000000 00000 10000 01010 01000 000000`.

`Shift Right Arithmetic (SRA)` uses the MSB as the shift-in bit (sign-extending):

Example (Shift right arithmetic).

`SRA $t2, $s0, 8`

Encoding: `000000 00000 10000 01010 01000 000011`.

Remark. Design questions: why is there no `SLA` (shift-left-arithmetic)? Why is there no `NOT`?

2.9.8 Jump register

Uses **register indirect addressing**.

- `JR rs`: jump register. `Rs` = target address; `Rd`, `Rt` = 0; `shamt` is a hint field (see the MIPS instruction set reference).
- `JALR rs` / `JALR rd, rs`: jump and link register. `Rs` = target address; `Rd` = return address; `Rt` = 0; `shamt` is a hint field.

2.9.9 I-Format instructions

Opcode	Rs	Rt	16-bit immediate value

This format is used for:

- Operation with immediate addressing.
- Operation with displacement addressing.
- Operation with PC-relative addressing.

Example (Immediate arithmetic and logical). `Op`: opcode; `Rs`: source register; `Rt`: destination register; `Constant`: immediate value (-32768 to 32767).

```
ADDI $t0, $t1, 0x0005
ORI  $t0, $t1, 0xFF00
```

Encodings:

```
001000 01001 01000 0000000000000101
001101 01001 01000 1111111100000000
```

(Design question: why is there no `SUBI`?)

Example (Load-Store (displacement)). `Rs`: base register number; 16-bit immediate value: offset added to the base address in `Rs`. Effective address $EA = Rs + 16\text{-bit immediate value}$. For loads, `Rt` is the destination register; for stores, `Rt` is the value stored to the `EA` in memory.

```
LW $t0, 16($t1)
SW $t0, 16($t1)
```

Encodings:

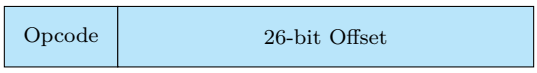
```
100011 01001 01000 0000000000010000
101011 01001 01000 0000000000010000
```

Example (PC-relative (near branches)). `Rs`, `Rt`: source register numbers; target address = `PC` + offset $\times 4$ (PC already incremented by 4 at this time).

```
BEQ $s0, $s1, 256    # if ($s0 == $s1) goto PC + 256*4;
```

Encoding: `000100 10000 10001 0000000100000000`.

2.9.10 J-Format instructions



Used for `J` and `JAL` – **pseudo-direct addressing**. Cannot fit a 32-bit value in the instruction, so target address = $PC_{31\ldots28} \,\|\, (\text{26-bit offset} \times 4)$.

Example (Jump).

```
J 0x01000000
```

Encoding: `000010 01000000000000000000000000`.

2.9.11 Moving data without a MOV instruction

Example (Move without MOV). How to move data from `$t0` to `$t1` using one MIPS instruction? MIPS has no dedicated `MOV`, so it is synthesized:

```
MOV $t1, $t0 ≡ add $t1, $t0, $zero
```

2.9.12 Atomic operations

Definition 2.9 (Load linked / store conditional). An atomic Read-Modify-Write operation is built from a pair of instructions: `LL` (Load Linked Word) and `SC` (Store Conditional Word).

```
LL $Rt, offset($Rs)
SC $Rt, offset($Rs)
```

If the memory word addressed by `LL` is modified before the matching `SC` to the same address, `SC` *fails* and returns 0 in `$Rt`. Otherwise, `SC` stores `$Rt` to memory and returns 1 in `$Rt`.

Example (Atomic swap).

```
try:  ADD $t0, $zero, $s4   # $t0 = $s4
      LL  $t1, 0($s1)       # $t1 = mem($s1)
      SC  $t0, 0($s1)       # mem($s1) = $t0 (if uncontended)
      BEQ $t0, $zero, try   # if mem($s1) changed meanwhile, retry
                             # else mem($s1) = old $t0
      ADD $s4, $zero, $t1   # $s4 = $t1
```

2.9.13 Constant (immediate) values

Small constants are used quite frequently ($\approx 50\%$ of operands in many common programs):

Example (Small and large constants).

```
# $t0 = 0x1234
ADDI $t0, $zero, 0x1234
```

For a 32-bit constant that does not fit in the 16-bit immediate field, use `LUI` (Load Upper Immediate) followed by `ORI`:

```
# $t0 = 0x12345678
LUI $t0, 0x1234
ORI $t0, $t0, 0x5678
```

2.9.14 Procedure call convention

Definition 2.10 (Procedure call convention).

1. Save the return address.
2. Save necessary registers.
3. Callee executes the function.
4. Restore previously saved registers.
5. Restore the return address.

6. Jump to the return address.

- `JAL label`: jump to a label (procedure); the return address is stored in `$ra` (register 31).
- `JR`: jump to the address stored in a register.

Example (Array element swap).

```
swap(int v[], int k)
{
    int temp;
    temp   = v[k];
    v[k]   = v[k+1];
    v[k+1] = temp;
}
```

Assume `swap` is called as a procedure; `temp` is register `$15`; arguments in `$a1`, `$a2`; `$16` is a scratch register.

```
swap:
    addi $sp,$sp,-4      ; create space on stack
    sw   $16, 0($sp)     ; callee-saved register put onto stack
    sll  $t2, $a2,2      ; multiply k by 4
    addu $t2, $a1,$t2    ; address of v[k]
    lw   $15, 0($t2)     ; load v[k]
    lw   $16, 4($t2)     ; load v[k+1]
    sw   $16, 0($t2)     ; store v[k+1] into v[k]
    sw   $15, 4($t2)     ; store old value of v[k] into v[k+1]
    lw   $16, 0($sp)     ; callee-saved register restored from stack
    addi $sp,$sp, 4      ; restore top of stack
    jr   $31             ; return to place that called swap
```

Example (Recursive factorial).

```
fact:
    addi $sp, $sp, -8   # adjust stack for 2 items
    sw   $ra, 4($sp)    # save return address
    sw   $a0, 0($sp)    # save argument

    slti $t0, $a0, 1    # test for n < 1
    beq  $t0, $zero, L1

    addi $v0, $zero, 1  # if so, result is 1
    addi $sp, $sp, 8     #   pop 2 items from stack
    jr   $ra             #   and return

L1: addi $a0, $a0, -1   # else decrement n
    jal  fact             # recursive call

    lw   $a0, 0($sp)    # restore original n
    lw   $ra, 4($sp)    #   and return address
    addi $sp, $sp, 8    # pop 2 items from stack

    mul  $v0, $a0, $v0  # multiply to get result n*fact(n-1)
    jr   $ra             # and return
```

2.9.15 Reference tables

Instruction	Example	Meaning	Comment
add	`add $1,$2,$3`	$1 = $2 + $3	3 operands; exception possible
subtract	`sub $1,$2,$3`	$1 = $2 – $3	3 operands; exception possible
add immediate	`addi $1,$2,100`	$1 = $2 + 100	+ constant; exception possible
add unsigned	`addu $1,$2,$3`	$1 = $2 + $3	3 operands; no exceptions
subtract unsigned	`subu $1,$2,$3`	$1 = $2 – $3	3 operands; no exceptions
add imm. unsign.	`addiu $1,$2,100`	$1 = $2 + 100	+ constant; no exceptions
multiply	`mult $2,$3`	Hi,Lo = $2 × $3	64-bit signed product
multiply unsigned	`multu $2,$3`	Hi,Lo = $2 × $3	64-bit unsigned product
divide	`div $2,$3`	Lo=$2÷$3, Hi=$2 mod $3	Lo=quotient, Hi=remainder
divide unsigned	`divu $2,$3`	Lo=$2÷$3, Hi=$2 mod $3	unsigned quotient & remainder
move from Hi	`mfhi $1`	$1 = Hi	used to get copy of Hi
move from Lo	`mflo $1`	$1 = Lo	used to get copy of Lo

Instruction	Example	Meaning	Comment
and	`and $1,$2,$3`	$1=$2&$3	3 reg. operands; logical AND
or	`or $1,$2,$3`	$1=$2|$3	3 reg. operands; logical OR
xor	`xor $1,$2,$3`	$1=$2⊕$3	3 reg. operands; logical XOR
nor	`nor $1,$2,$3`	$1=∼($2|$3)	3 reg. operands; logical NOR
and immediate	`andi $1,$2,10`	$1=$2&10	logical AND reg, constant
or immediate	`ori $1,$2,10`	$1=$2|10	logical OR reg, constant
xor immediate	`xori $1,$2,10`	$1=∼$2&∼10	logical XOR reg, constant
shift left logical	`sll $1,$2,10`	$1=$2«10	shift left by constant
shift right logical	`srl $1,$2,10`	$1=$2»10	shift right by constant
shift right arithm.	`sra $1,$2,10`	$1=$2»10	shift right (sign extend)
shift left logical (var)	`sllv $1,$2,$3`	$1=$2«$3	shift left by variable
shift right logical (var)	`srlv $1,$2,$3`	$1=$2»$3	shift right by variable
shift right arithm. (var)	`srav $1,$2,$3`	$1=$2»$3	shift right arith. by variable

Instruction	Comment
`SW 500(R4), R3`	Store word
`SH 502(R2), R3`	Store half
`SB 41(R3), R2`	Store byte
`LW R1, 30(R2)`	Load word
`LH R1, 40(R3)`	Load halfword
`LHU R1, 40(R3)`	Load halfword unsigned
`LB R1, 40(R3)`	Load byte
`LBU R1, 40(R3)`	Load byte unsigned
`LUI R1, 40`	Load upper immediate (16 bits shifted left by 16)

Compare and Branch	`BEQ rs,rt,offset`	if R[rs]==R[rt] then PC-relative branch
	`BNE rs,rt,offset`	…not-equal ($\neq$)
	`BLEZ rs,offset`	if R[rs] $\leq$ 0 then PC-relative branch
	`BGTZ rs,offset`	…greater-than ($>$)
	`BLTZ rs,offset`	…less-than ($<$)
	`BGEZ rs,offset`	…greater-or-equal ($\geq$)

Instruction	Example	Meaning
branch on equal	`beq $1,$2,100`	if ($1==$2) go to PC+4+100; equal test; PC-relative branch
branch on not eq.	`bne $1,$2,100`	if ($1!=$2) go to PC+4+100; not-equal test; PC-relative
set on less than	`slt $1,$2,$3`	if ($2<$3) $1=1; else $1=0; compare less than; 2's comp.
set less than imm.	`slti $1,$2,100`	if ($2<100) $1=1; else $1=0; compare < constant; 2's comp.
set less than uns.	`sltu $1,$2,$3`	if ($2<$3) $1=1; else $1=0; compare less than; natural no.
set l.t. imm. uns.	`sltiu $1,$2,100`	if ($2<100) $1=1; else $1=0; compare < constant; natural
jump	`j 10000`	go to 10000; jump to target address
jump register	`jr $31`	go to $31; for switch, procedure return
jump and link	`jal 10000`	$31 = PC+4; go to 10000; for procedure call

2.9.16 Assembler directives and example programs

Definition 2.11 (Assembler directives). Common assembler (GAS/SPIM-style) directives: `.text` (code section), `.data` (data section), `.globl` (export a label), `.word` (a 32-bit word constant), `.space` (reserve n bytes).

Example (Adding two constants).

```
.text
.globl main
main:
    li   $t0, 0x2          # $t0 <- 0x2
```

```
    li   $t1, 0x3          # $t1 <- 0x3
    addu $t2, $t0, $t1     # $t2 <- ADD($t0, $t1)
```

which is equivalent to (`li` pseudo-instruction expanded):

```
.text
.globl main
main:
    ori  $t0, $0, 0x2      # $t0 <- OR(0, 0x2)
    ori  $t1, $0, 0x3      # $t1 <- OR(0, 0x3)
    addu $t2, $t0, $t1     # $t2 <- ADD($t0, $t1)
```

Example (Using labeled data).

```
.data
n:  .word 0x2
m:  .word 0x3
r:  .space 4

.text
.globl main
main:
    lw   $t0, n            # load n to $t0
    lw   $t1, m            # load m to $t1
    addu $t2, $t0, $t1     # $t2 <- ADD($t0, $t1)
    sw   $t2, r            # store $t2 to r
```

Example (Indexing into an array (`la` + displacement)).

```
.data
n:  .word 0x2, 0x3, 0x4

.text
.globl main
main:
    la   $t5, n            # load address of n to $t5
    lw   $t0, 0($t5)       # load n[0] to $t0
    lw   $t1, 4($t5)       # load n[1] to $t1
    addu $t2, $t0, $t1     # $t2 <- ADD($t0, $t1)
    sw   $t2, 8($t5)       # store $t2 to n[2]
```

2.9.17 Using the SPIM simulator

A typical SPIM (QtSpim) workflow to assemble, run, and debug MIPS code:

1. **Start SPIM**: the window shows the integer/FP register file, the program's text segment (disassembly), and a message log.
2. **Load a program**: *File* → *Reinitialize and Load File*, then select your assembly (`.s`) file.
3. **Execute**: step or run; the current instruction is highlighted, and register changes from the previous instruction are shown.
4. **Inspect data**: the *Data* tab shows the user data segment and stack contents in memory.

5. **Set a breakpoint**: right-click an instruction in the text segment and choose *Set Breakpoint* (e.g. at a conditional branch).
6. **Single-step**: use the step button (F10) to execute the program line by line and watch register/memory state evolve.

2.9.18 Reference

MIPS instruction set reference, course handout (`references/mips-reference-data.pdf`); see also the reformatted version, `references/mips-reference-readable.pdf`.

2.10 Exercises

These are left unresolved for self-study.

Exercise 2.12 (Procedure-call convention).

(1) Which one, caller or callee, has to save the return address?

(2) Which one, caller or callee, has to save necessary (callee-saved) registers?

(3) Is it necessary for a leaf procedure to save the return address (in MIPS)? Justify your answer.

Exercise 2.13 (Review quiz). Trong hợp ngữ MIPS, có thể thực hiện phép toán `NOT` bằng:

a. Lệnh `NOT`

b. Lệnh `NOR`

c. Lệnh `NAND`

d. Tất cả đều sai

Exercise 2.14 (Review quiz). Chế độ định địa chỉ (addressing mode) là:

a. Cách tham khảo đến toán hạng/ lệnh (nếu là lệnh nhảy)

b. Cách tổ chức địa chỉ bộ nhớ theo byte/halfword/word

c. Cách sắp xếp thứ tự bộ nhớ (endianness)

d. Tất cả đều đúng

Exercise 2.15 (Review quiz). Thanh ghi `LO`, `HI` trong MIPS dùng để:

a. Tính địa chỉ thấp, cao

b. Thao tác phép nhân, chia

c. Cả a và b đều đúng

d. Cả a và b đều sai

Exercise 2.16 (Review quiz). Kiến trúc MIPS là kiến trúc có kiểu lưu trữ:

a. Little-Endian

b. Big-Endian

c. Hỗ trợ cả hai kiểu trên

d. Không hỗ trợ Endian

Exercise 2.17 (C-to-MIPS translation). Consider the following C code fragment:

```
if (a == 0) {
    a = 1;
} else {
    a = 0;
}
```

Q10) Which of the following MIPS instruction sequences performs the *same* operation as the given code fragment? (Select all that apply, and justify why the others are wrong.)

(A)

```
lw   $t0,a
bne  $t0,$0,L1
addi $t0,$0,1
b    L2
L1: add $t0,$0,$0
L2: sw   $t0,a
```

(B)

```
lw   $t0,a
beq  $t0,$0,L1
sw   $0,a
b    L2
L1: addi $t0,$0,1
    sw   $t0,a
L2:
```

(C)

```
lw    $t0,a
sltiu $t0,$t0,1
sw    $t0,a
```

(D)

```
lw   $t0,a
addi $t1,$0,1
beq  $t0,$0,L1
addi $t1,$0,0
L1: sw   $t1,a
```

(E) All of the above

2.11 Summary

- The ISA is the interface between software and hardware; ISA styles differ in where operands live (stack, accumulator, memory, registers). Modern ISAs are load-store (register-register).
- Instruction set design covers operations, addressing modes, operand types, encoding (fixed vs. variable length), and the register file. Immediate and displacement addressing dominate real programs.
- CISC packs more work per instruction; RISC keeps instructions simple and regular to enable pipelining and CPI ≈ 1.
- MIPS: 32 registers, three 32-bit formats (R, I, J), big-endian, load-store, with a register convention that splits caller-/callee-saved registers for procedure calls.

3 Lecture 3: Pipelining

September 27, 2026

3.1 Overview

Pipelining overlaps the execution of several instructions so that, ideally, one instruction completes every clock cycle. This lecture builds the classic 5-stage MIPS pipeline, quantifies its speedup, and then studies what breaks the ideal: structural, data, and control hazards, together with the hardware (forwarding, interlocks, early branch resolution) and compiler (scheduling, delay slots) techniques that reduce their cost.

3.2 What is Pipelining?

Definition 3.1 (Pipelining)**.** **Pipelining** is an implementation technique in which multiple instructions are overlapped in execution.

Real-life pipelining examples:

- Doing laundry (washing, drying, folding multiple loads)
- Factory production lines
- Traffic (?)

3.2.1 Instruction Pipelining

Definition 3.2 (Instruction pipeline, stage, machine cycle)**.**

- Instruction pipelining is a CPU implementation technique where **multiple operations** on a number of instructions are **overlapped**.
- An instruction execution pipeline involves a number of steps, where each step completes a part of an instruction. Each step is called a *pipeline stage* or *pipeline segment*.
- The stages are connected in a **linear fashion**: one stage to the next, forming the pipeline – instructions enter at one end, progress through the stages, and exit at the other end.
- The time to move an instruction one step down the pipeline is equal to *the machine cycle* and is determined by the stage with the longest processing delay.

- Pipelining **increases** the CPU **instruction throughput**: the number of instructions completed per cycle.
  - Under ideal conditions (no stall cycles), instruction throughput is one instruction per machine cycle, i.e. ideal CPI = 1.
- Pipelining **does not reduce** the execution time of an individual instruction: the time needed to complete all processing steps of an instruction (also called *instruction completion latency*).
  - Minimum instruction latency = n cycles, where n is the number of pipeline stages.

Explanation. **Why is the ideal CPI equal to 1?** CPI counts how many cycles pass, on average, between *completions* of instructions – not how long each instruction takes. In a k-stage pipeline each instruction still needs k cycles from fetch to write-back, but once the

pipeline is full a new instruction enters and an old one leaves every cycle. For n instructions the pipeline needs $k + (n - 1)$ cycles, so

$$\text{CPI} = \frac{k + n - 1}{n} \longrightarrow 1 \quad (n \to \infty).$$

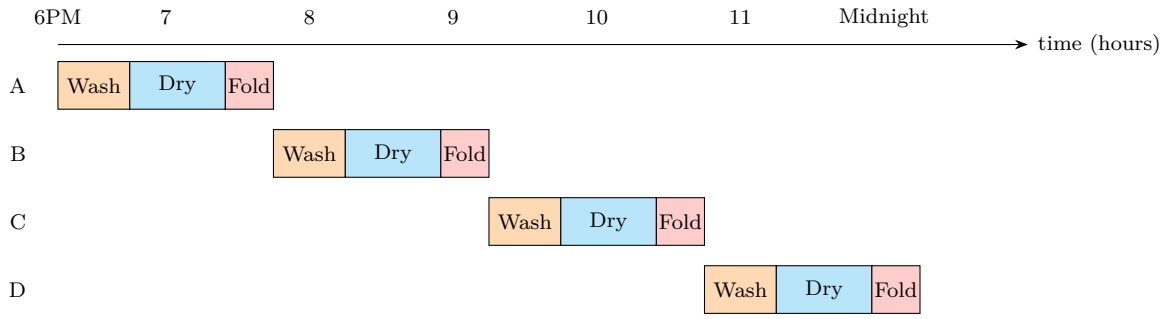
The fill cost of $k - 1$ cycles is paid only once. Stalls caused by hazards add cycles without completing instructions, which is why real pipelines have CPI = 1+stall cycles per instruction.

3.3 Pipelining Example: Laundry

Example (Doing laundry). Ann, Brian, Cathy, and Dave (A, B, C, D) each have one load of clothes to wash, dry, and fold.

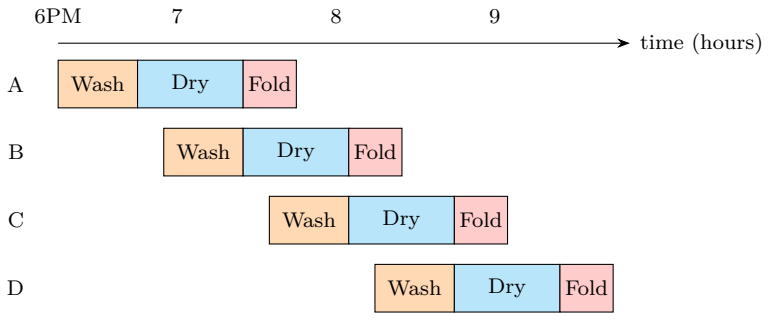
- Washer takes 30 minutes
- Dryer takes 40 minutes
- "Folder" takes 20 minutes

3.3.1 Sequential laundry



Remark. **Sequential laundry takes 6 hours for 4 loads** (each load: $30 + 40 + 20 = 90$ min, done strictly one after another).

3.3.2 Pipelined laundry (start work ASAP)



Remark. **Pipelined laundry takes 3.5 hours for 4 loads.**

$$\text{Speedup} = \frac{6}{3.5} = 1.7$$

3.3.3 Pipelining lessons

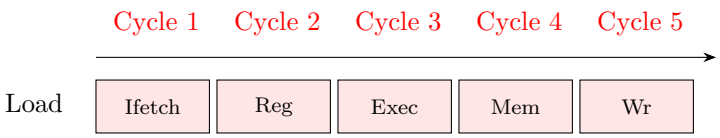
Pipelining lessons

- Pipelining does not help the **latency** of a single task; it helps the **throughput** of the entire workload.
- Pipeline rate is limited by the **slowest** pipeline stage.
- Multiple tasks operate **simultaneously**, using **different resources**.
- Potential speedup = number of pipe stages.
- Unbalanced lengths of pipe stages reduce speedup (dryer is the bottleneck at 40 min ⇒ every stage effectively takes 40 min).
- Time to "fill" the pipeline and time to "drain" it reduces speedup.
- As long as we have **separate resources**, we can pipeline the tasks.
- If one load depends on another, we have to wait (delay/stall for *dependencies*).

3.4 CPU Pipelining

3.4.1 The 5 classic MIPS stages

1. **IF** – Fetch instruction from instruction memory
2. **ID** – Read registers while decoding the instruction (Reg/Dec)
3. **EX** – Execute the operation or calculate an address, depending on instruction type
4. **MEM** – Access an operand from data memory
5. **WB** – Write the result into a register



Resources needed by a `lw` (load) instruction at each stage:

Stage	Resource
Ifetch (IF)	Instruction memory (IM)
Reg/Dec (ID)	Register file & decoder (Reg)
Exec (EX)	ALU
Mem (MEM)	Data memory (DM)
Wr (WB)	Register file (Reg)

Note. Accessing source and destination registers happens in *two different halves* of the clock cycle:

- **Read** occurs in the 2nd half of the ID cycle.
- **Write** occurs in the 1st half of the WB cycle.

This lets an instruction read a register in the same cycle that an earlier instruction writes it, and get the correctly updated value.

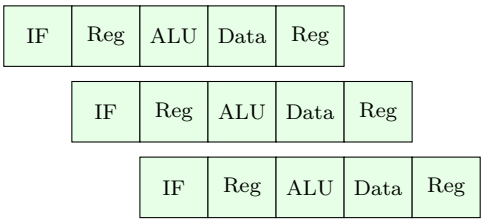
3.4.2 Single-cycle vs. pipelined execution – example

Consider three loads: `lw $1,100($0)`, `lw $2,200($0)`, `lw $3,300($0)`. Each of the 5 stages below is 2 ns (limited by the slowest resource); a non-pipelined machine needs one long 8 ns cycle per instruction.

Single-cycle, non-pipelined: total time for 3 instructions = 3 × 8ns = 24 ns.



Single-cycle, pipelined (each stage clocked at 2 ns; write to register in 1st half of clock, read in 2nd half): total time for 3 instructions = 14 ns. Each additional instruction adds only 2 ns.



Operation times used throughout these notes:

Instr. class	Instr. fetch	Reg. read	ALU op	Data access	Reg. write	Total
lw	2ns	1ns	2ns	2ns	1ns	8ns
sw	2ns	1ns	2ns	2ns		7ns
add, sub, and, or, slt	2ns	1ns	2ns		1ns	6ns
beq	2ns	1ns	2ns			5ns

(Clock cycle must accommodate the slowest instruction ⇒ 2ns.)

3.4.3 Theoretical speedup

Theorem 3.3 (Ideal pipeline speedup). For n tasks through a k-stage pipeline with *balanced* stages, let p be the time to complete one task *without* pipelining (so each individual pipeline stage takes p/k). Non-pipelined, n tasks take $n \cdot p$; pipelined, the first task takes p to fill the pipeline and each of the remaining $n-1$ tasks adds one stage-time p/k:

$$\text{Speedup} = \frac{n \cdot p}{\frac{p}{k}(n-1)+p} = \frac{nk}{n+k-1} \approx k \quad \text{(for large } n\text{)}$$

Proof. Non-pipelined, each task takes p, so n tasks take np. Pipelined, the first task needs all k stages ($k \cdot p/k = p$), and each later task finishes one stage-time p/k after the previous one. Divide, and note that $\frac{p}{k}(n-1)+p = \frac{p}{k}(n+k-1)$. □

Practically, speedup is *less than* the number of stages because:

- Stages are imperfectly balanced.
- Pipelining needs overhead (extra latches, control logic).

Example (Three vs. 1003 loads)**.** 3 consecutive instructions: non-pipelined $8 \times 3 = 24$ns, pipelined 14ns $\Rightarrow$ speedup $= 24/14 = 1.7$. For 1003 consecutive instructions: non-pipelined $= 1000 \times 8 + 24 = 8024$ns; pipelined $= 1000 \times 2 + 14 = 2014$ns $\Rightarrow$ speedup $\approx 3.98 \approx 8$ns/2ns – near-perfect speedup. Performance increases with the number of instructions (throughput dominates).

3.4.4 Why MIPS is easy to pipeline

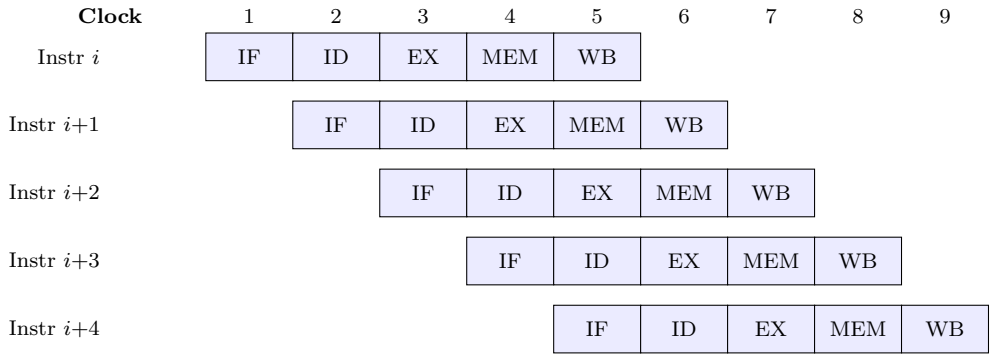
MIPS was designed with pipelining in mind:

1. **All instructions are the same length.** Fetch in stage 1, decode in stage 2. If instruction length varies (e.g. 80x86), pipelining is more challenging.
2. **Limited instruction formats**, with source registers always in the same place $\Rightarrow$ can read the register file before decoding the instruction.
3. **Memory operands appear only in `lw` & `sw`.** We can use the Execute stage to calculate the memory address and access memory in the next stage. In a machine that could operate on memory operands directly (e.g. 80x86), the Execute/Memory stages would expand into: address calculation, memory access, execute.
4. **Operands must be aligned in memory**, so a single data transfer needs only one memory access.

3.5 MIPS In-Order Single-Issue Integer Pipeline

3.5.1 Ideal pipeline operation (no stalls)

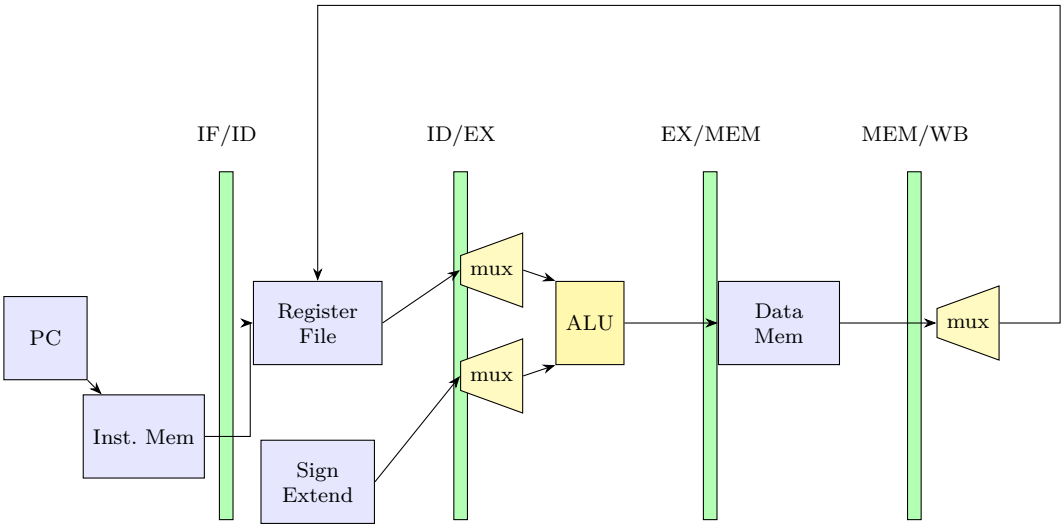
Fill cycles = number of stages -1. With $n = 5$ stages, ideal CPI $= 1$.



IF = Instruction Fetch, ID = Instruction Decode, EX = Execution, MEM = Memory Access, WB = Write Back. *In-order* = instructions execute in original program order. *Ideal* = pipeline operation without any stall cycles.

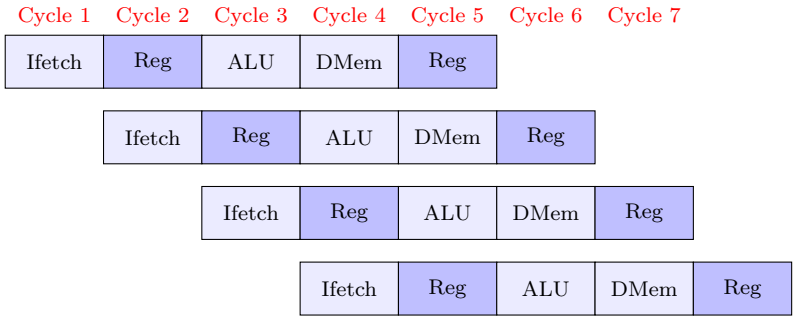
3.5.2 5 steps of the MIPS datapath (with pipeline registers)

Data is carried from one stage to the next in **pipeline registers/latches** (IF/ID, ID/EX, EX/MEM, MEM/WB), which hold temporary values between clocks and the info needed for later stages.



Note: this is a *data-stationary* control scheme – each pipeline stage locally decodes the control signals it needs from the instruction bits carried along in the pipeline registers.

3.5.3 Visualizing the pipeline (register read/write timing)



- Read operand registers in the **second half** of the ID cycle.
- Write the destination register in the **first half** of the WB cycle.

This is an ideal integer in-order 5-stage pipeline.

3.5.4 Pipelining performance example

Example (Pipelining a multi-cycle CPU). For an unpipelined CPU: clock cycle = 1ns; 4 cycles for ALU operations/branches, 5 cycles for memory operations; instruction mix 40%/20%/40% (ALU/branch/memory). Pipelining adds 0.2ns to the clock cycle.

$$\text{Non-pipelined avg. instr. time} = 1\text{ns} \times ((40\% + 20\%) \times 4 + 40\% \times 5) = 4.4\text{ns}$$

$$\text{Pipelined avg. instr. time} = 1\text{ns} + 0.2\text{ns} = 1.2\text{ns}$$

$$\text{Speedup} = \frac{4.4}{1.2} = 3.7\times \text{ faster}$$

3.6 Pipeline Hazards

Definition 3.4 (Pipeline hazards). **Hazards** are situations in pipelining that prevent the next instruction in the instruction stream from executing during its designated clock cycle,

possibly resulting in one or more *stall* (wait) cycles. Hazards increase CPI above 1 and are classified into three types:

- **Structural hazards**: arise from hardware resource conflicts when available hardware cannot support all possible combinations of instructions.
- **Data hazards**: arise when an instruction depends on the result of a previous instruction in a way exposed by the overlapping of instructions in the pipeline.
- **Control hazards**: arise from the pipelining of conditional branches and other instructions that change the PC.

Note. An instruction "issued later" than instruction x was issued after x and is *not as far along* in the pipeline. An instruction "issued earlier" than x was issued before x and is *further along* in the pipeline.

3.6.1 Stalls and performance

Stalls impede progress of a pipeline and cause deviation from 1 instruction/cycle.

Proposition 3.5 (CPI and speedup with stalls)**.**

$$\text{CPI}_{\text{pipelined}} = \text{Ideal CPI} + \text{Pipeline stall cycles per instruction} = 1 + \text{stall cycles/instr.}$$

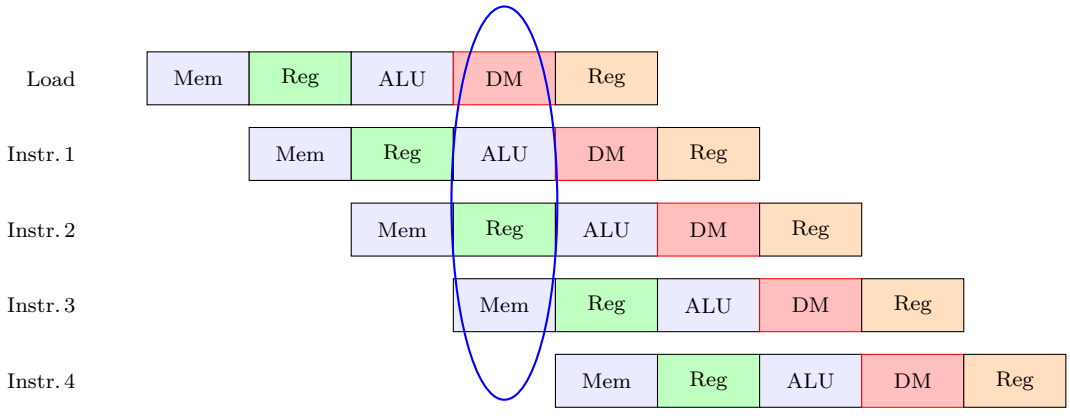
Ignoring overhead, assuming balanced stages:

$$\text{Speedup} = \frac{\text{CPI}_{\text{unpipelined}}}{1 + \text{pipeline stall cycles per instruction}}$$

3.7 Structural Hazards

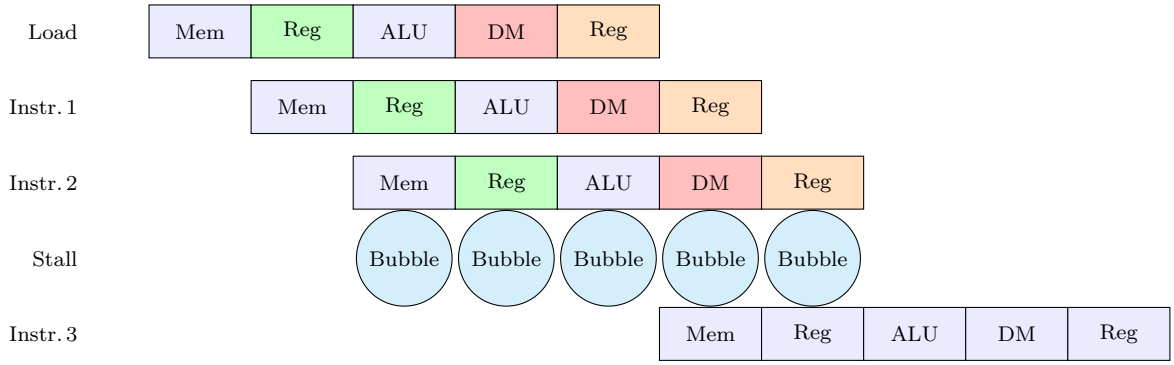
Definition 3.6 (Structural hazard)**.** In pipelined machines, overlapped instruction execution requires pipelining of functional units and duplication of resources so all possible instruction combinations can be supported. If a **resource conflict** arises because a hardware resource is required by more than one instruction in a single cycle, and it cannot be accommodated, a **structural hazard** occurs – e.g. a pipelined machine with a single shared memory port for instructions and data ⇒ stall the pipeline for one cycle for a memory data access.

3.7.1 Example: single shared memory port



conflict: all instructions want DM in the same cycle

Resolution: stall (insert a bubble/NOP).



3.7.2 Structural hazard: numeric example

Example (Is a structural hazard worth it?). Given data references are 40% of a program, ideal pipelined CPI = 1. A machine with a data-memory structural hazard needs 1 stall cycle per data reference, but its clock rate is 1.05× higher than the ideal machine:

$$\text{Avg. instr. time} = (1 + 0.4 \times 1) \times \frac{\text{Clock}_{\text{ideal}}}{1.05} = 1.3 \times \text{Clock}_{\text{ideal}}$$

$\Rightarrow$ the machine *without* the hazard is still better ($1.3 > 1$).

Remark. Remember the common case! All else equal, a machine without structural hazards always has lower CPI, but sometimes it is cheaper (in hardware) to allow a hazard than to eliminate it – weigh HW cost of pipelining/duplicating a unit against how often the hazard actually occurs.

3.8 Data Hazards

Definition 3.7 (Data hazard). Data hazards occur when the pipeline changes the order of read/write accesses to operands compared to the original sequential order, potentially causing **incorrect execution**. One or more instructions may need to stall to ensure correctness.

Example (RAW on R1).

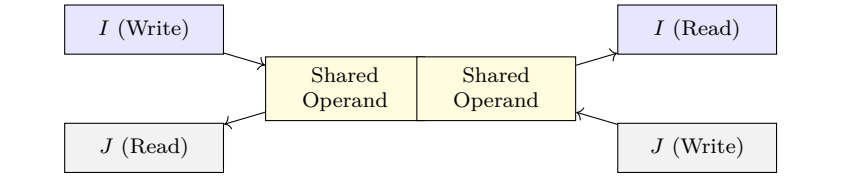
```
ADD R1, R2, R3
SUB R4, R1, R5
AND R6, R1, R7
OR  R8, R1, R9
XOR R10, R1, R11
```

All instructions after `ADD` use its result (`R1`); `SUB` and `AND` would need to stall for correct execution without forwarding.

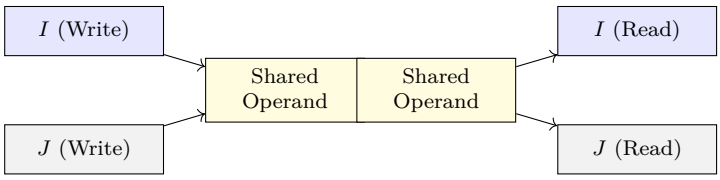
3.8.1 Data Hazard Classification

Definition 3.8 (RAW, WAW, WAR). Given two instructions I, J, with I occurring before J in program order:

- **RAW** (read after write) – *true data dependence*: J tries to read a source before I writes it, so J incorrectly gets the old value.
- **WAW** (write after write) – *name dependence*: J tries to write an operand before it is written by I; the writes end up performed in the wrong order.
- **WAR** (write after read) – *name dependence*: J tries to write a destination before it is read by I, so I incorrectly gets the new value.
- **RAR** (read after read): *not* a hazard.



RAW (true dependence) **WAR (name dependence)**

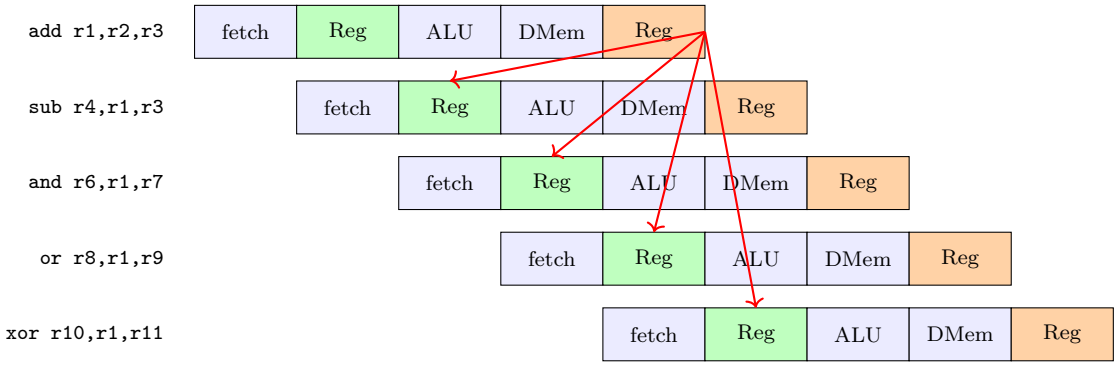


WAW (name dependence) **RAR (not a hazard)**

- **RAW example:** `i: ADD R1,R2,R3  j: SUB R4,R1,R6`
- **WAW example:** `i: SUB R1,R4,R3  j: ADD R1,R2,R3`
- **WAR example:** `i: SUB R4,R1,R3  j: ADD R1,R2,R3`

Remark. RAW and WAW can be resolved with stalling or forwarding; WAR does not occur in the simple MIPS pipeline because reads happen early (ID) and writes late (WB) in program order, so instructions never write before an earlier one has read (only relevant for pipelines that reorder execution).

3.8.2 Data hazard on R1 (visualized)



Without forwarding, `add`'s result is only available (written back) after its WB stage, but `sub` needs `r1` in its ID stage one cycle earlier – a RAW hazard.

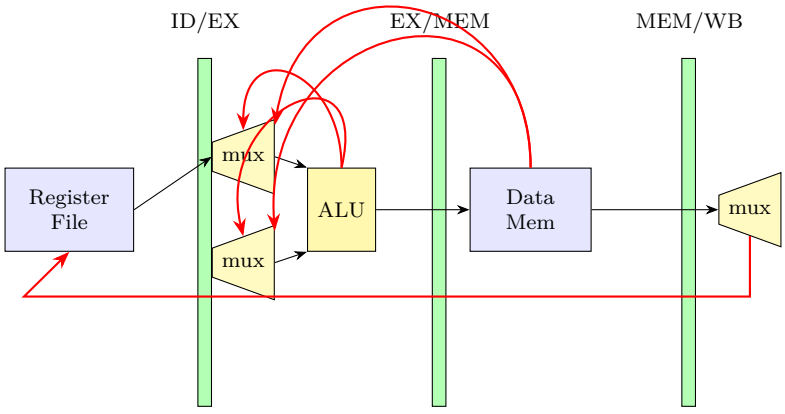
3.9 Minimizing Data Hazard Stalls by Forwarding

Definition 3.9 (Forwarding). **Forwarding** (a.k.a. *register bypassing* or *short-circuiting*) is a hardware-based technique used to eliminate or minimize data-hazard stalls. The result of an instruction is copied directly from where it is produced (ALU output, memory read port, etc.) to where a subsequent instruction needs it (ALU input register, memory write port, etc.), *without* waiting for it to be written to and re-read from the register file.

In the MIPS integer pipeline with forwarding:

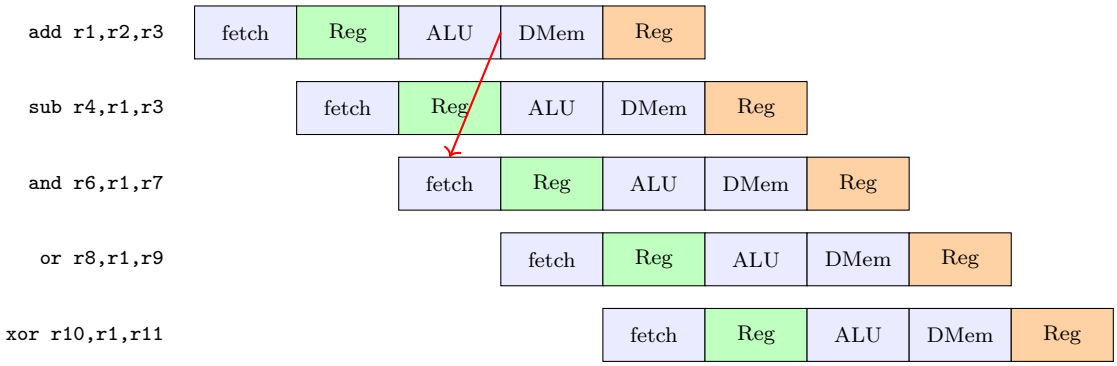
- The **ALU result** from the EX/MEM register may be forwarded back to the ALU input latches instead of the register value read in ID.
- The **Data Memory result** from the MEM/WB register may also be forwarded to the ALU input latches.
- If the forwarding hardware detects that a previous ALU operation will write the register that is a source for the current ALU operation, control logic selects the forwarded result as the ALU input instead of the value read from the register file.

3.9.1 Datapath modified for forwarding



Red paths show the EX/MEM → ALU-input and MEM/WB → ALU-input forwarding routes, plus the normal write-back path to the register file.

3.9.2 Forwarding to avoid the RAW hazard



The ALU result of `add` is forwarded directly into the ALU input of `sub` in the very next cycle – no stall needed.

3.9.3 Forwarding to avoid an LW–SW data hazard

```
add r1,r2,r3
lw  r4,0(r1)
sw  r4,12(r1)
or  r8,r6,r9
xor r10,r9,r11
```

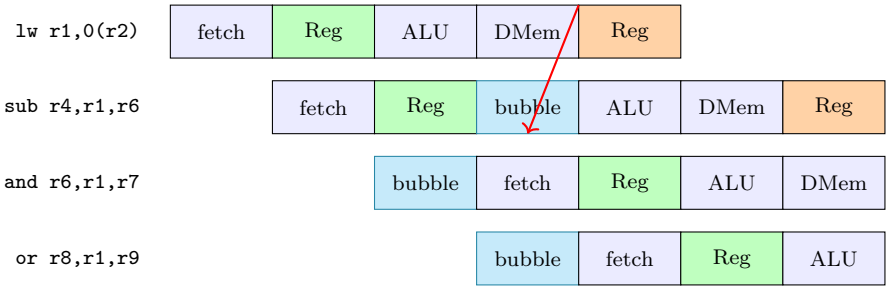
> **Note.** The value loaded by `lw` into `r4` is forwarded from the MEM/WB register directly to the `sw`'s store-data input, avoiding a stall.

3.10 Data Hazard Even With Forwarding: the Load-Use Hazard

> **Remark.** Some hazards *cannot* be resolved by forwarding alone, because the needed value is not produced early enough.

```
lw  r1, 0(r2)
sub r4, r1, r6
and r6, r1, r7
or  r8, r1, r9
```

`lw` produces `r1` at the *end of its MEM stage* (cycle 4), but `sub` needs it as an ALU input at the *start of its EX stage* (cycle 3) – one cycle too early even with forwarding. A single stall cycle (bubble) must be inserted:



3.10.1 Hardware pipeline interlocks

Definition 3.10 (Pipeline interlock)**.** A **hardware pipeline interlock** detects a data hazard and stalls the pipeline until the hazard clears. The CPI for the stalled instruction increases by the length of the stall.

Listing 1: No hazard (ideal)

```
              c1    c2    c3    c4    c5    c6    c7    c8    c9
LW  R1,0(R1)  IF    ID    EX    MEM   WB
SUB R4,R1,R5        IF    ID    EX    MEM   WB
AND R6,R1,R7              IF    ID    EX    MEM   WB
OR  R8,R1,R9                    IF    ID    EX    MEM   WB
```

Listing 2: Load-use hazard: interlock inserts one stall, then forwards

```
              c1    c2    c3    c4    c5    c6    c7    c8    c9
LW  R1,0(R1)  IF    ID    EX    MEM   WB
SUB R4,R1,R5        IF    ID    stall EX    MEM   WB
AND R6,R1,R7              IF    stall ID    EX    MEM   WB
OR  R8,R1,R9                    stall IF    ID    EX    MEM   WB
```

3.10.2 Data hazards and the compiler

Definition 3.11 (Pipeline (instruction) scheduling)**.** The compiler can help eliminate some hazard-induced stalls: it should avoid generating a `LOAD` instruction immediately followed by an instruction that uses the load's destination register. This technique is called **pipeline / instruction scheduling**.

Situation	Example	Action

No dependence	`LW R1,45(R2)`
	`ADD R5,R6,R7`
	`SUB R8,R6,R7`
	`OR R9,R6,R7`
		No hazard possible: no dependence on R1 in the following three instructions.

Dependence requiring stall	`LW R1,45(R2)`
	`ADD R5,R1,R7`
	`SUB R8,R6,R7`
	`OR R9,R6,R7`
		Comparators detect the use of R1 in ADD and stall ADD (and SUB, OR) before ADD begins EX.

Dependence overcome by forwarding	`LW R1,45(R2)`
	`ADD R5,R6,R7`
	`SUB R8,R1,R7`
	`OR R9,R6,R7`
		Comparators detect the use of R1 in SUB and forward the LOAD result to the ALU in time for SUB's EX.

Dependence with accesses in order	`LW R1,45(R2)`
	`ADD R5,R6,R7`
	`SUB R8,R6,R7`
	`OR R9,R1,R7`
		No action required: the read of R1 by OR occurs in the 2nd half of ID, after the write of the loaded data in the 1st half.

3.10.3 Static compiler instruction scheduling example

Note. *Static* = at compilation time, by the compiler. *Dynamic* = at run time, by hardware in the CPU.

Example (Scheduling `a = b + c; d = e - f;`). For `a = b + c; d = e - f;` (a,b,c,d,e,f in memory), assuming loads have a latency of 1 cycle:

Original code (2 stalls):

```
LW  Rb, b
LW  Rc, c
ADD Ra,Rb,Rc  (*stall*)
SW  Ra, a
LW  Re, e
LW  Rf, f
SUB Rd,Re,Rf  (*stall*)
SW  Rd, d
```

Scheduled code (no stalls):

```
LW  Rb, b
LW  Rc, c
LW  Re, e
ADD Ra,Rb,Rc
LW  Rf, f
SW  Ra, a
SUB Rd,Re,Rf
SW  Rd, d
```

3.10.4 Performance of pipelines with stalls

$$\text{CPI}_{\text{pipelined}} = \text{Ideal CPI} + \text{pipeline stall cycles/instr.} = 1 + \text{stall cycles/instr.}$$

$$\text{Speedup} = \frac{\text{CPI}_{\text{unpipelined}}}{1 + \text{pipeline stall cycles/instr.}}$$

When all instructions in the multi-cycle CPU take the same number of cycles (equal to the number of pipeline stages):

$$\text{Speedup} = \frac{\text{Pipeline depth}}{1 + \text{pipeline stall cycles/instr.}}$$

3.11 Control Hazards

Definition 3.12 (Control hazard). When a conditional branch executes, it may change the PC; without special measures this stalls the pipeline for a number of cycles until the branch condition is known (the *branch is resolved*) – otherwise the PC may not be correct when needed in IF.

In the (unmodified) MIPS pipeline, the conditional branch is resolved in stage 4 (MEM), giving **3 stall cycles**:

```
             c1    c2    c3    c4    c5    c6    c7    c8    c9    c10
Branch       IF    ID    EX    MEM   WB
Successor          stall stall stall IF    ID    EX    MEM   WB
Successor+1                          IF    ID    EX    MEM   WB
```

Proposition 3.13 (Branch penalty).

$$\text{Branch Penalty} = (\text{stage where branch is resolved}) - 1 = 4 - 1 = 3 \text{ cycles}$$

3.11.1 Control hazard on branches – three-stage stall (illustration)

```
12: beq r1,r3,36
16: and r2,r3,r5   -- squashed if branch taken
20: or  r6,r1,r7   -- squashed if branch taken
24: add r8,r1,r9   -- squashed if branch taken
36: xor r10,r1,r11 -- branch target
```

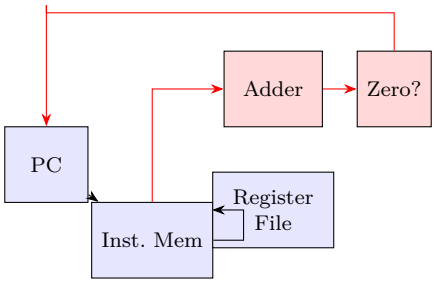
If the branch at PC 12 is taken, the three sequentially-fetched instructions at 16, 20, 24 must be discarded (squashed) once the branch outcome is known in MEM, and the target instruction at 36 is fetched instead.

3.11.2 Reducing branch stall cycles

Two hardware measures reduce the branch penalty:

1. Find out whether a branch is taken *earlier* in the pipeline.
2. Compute the taken-branch target PC *earlier* in the pipeline.

In MIPS, branch instructions (`BEQZ`, `BNE`, …) test a register for equality to zero. This zero-test can be moved into the ID cycle, and both PCs (taken and not-taken) computed early using an **additional adder** (the ALU is not usable until EX). This reduces the branch penalty to **just 1 stall cycle**.



Modified MIPS pipeline: conditional branch completed in the ID stage; branch penalty $= 2 - 1 = 1$ *cycle.*

3.11.3 Branch stall impact

Example (Impact of a 3-cycle branch stall). If CPI $= 1$ and 30% of instructions are branches with a 3-cycle stall: new CPI $= 1 + 0.30 \times 3 = 1.9$ – almost double! Reducing the penalty to 1 cycle (MIPS solution above) is therefore very valuable.

3.12 Four Branch Hazard Alternatives

3.12.1 #1: Stall until branch direction is clear

Simplest, but wastes cycles as shown above.

3.12.2 #2: Predict Branch Not Taken (most common)

- Execute successor instructions in sequence.
- "Squash" instructions in the pipeline if the branch is actually taken.
- Advantage: late pipeline state update (nothing irrevocable has happened yet).
- About 47% of MIPS branches are not taken on average.
- $PC + 4$ is already calculated, so it is used to fetch the next instruction.

Predict-not-taken timing (branch penalty $= 1$ **cycle, ID-stage resolution).**

Listing 3: Not taken (no stall)

```
                c1    c2    c3    c4    c5    c6    c7
Untaken branch  IF    ID    EX    MEM   WB
Instr i+1             IF    ID    EX    MEM   WB
Instr i+2                   IF    ID    EX    MEM   WB
```

Listing 4: Taken (1 idle cycle)

```
                c1    c2    c3    c4    c5    c6    c7
Taken branch    IF    ID    EX    MEM   WB
```

```
Instr i+1             IF    idle  idle  idle  idle
Branch target               IF    ID    EX    MEM   WB
```

Proposition 3.14 (Branch stall cycles).

Pipeline stall cycles from branches = frequency of taken branches × branch penalty

3.12.3 #3: Predict Branch Taken

- About 53% of MIPS branches are taken on average.
- But (in the basic MIPS pipeline) the branch target address hasn't been calculated yet, so MIPS still incurs a 1-cycle branch penalty even with this scheme (other machines that know the branch target before the outcome can benefit more).

3.12.4 #4: Delayed Branch

Definition 3.15 (Delayed branch). Define the branch to take place *after* a following instruction:

```
branch instruction
  sequential successor 1  <- branch delay slot(s)
  ...
  sequential successor n
branch target if taken
```

A single delay slot suffices for proper branch-decision and branch-target computation in a 5-stage pipeline. **MIPS uses this.**

Delayed branch timing (no stalls, whether or not taken).

```
                c1    c2    c3    c4    c5    c6    c7
Untaken branch  IF    ID    EX    MEM   WB
Delay slot i+1        IF    ID    EX    MEM   WB
Instr i+2             IF    ID    EX    MEM   WB

                c1    c2    c3    c4    c5    c6    c7
Taken branch    IF    ID    EX    MEM   WB
Delay slot i+1        IF    ID    EX    MEM   WB
Branch target               IF    ID    EX    MEM   WB
```

The behaviour of the delay-slot instruction is the same whether or not the branch is taken.

3.12.5 Scheduling branch delay slots

A. From before branch	B. From branch target	C. From fall-through

`add $1,$2,$3`
`if $2=0 then`
`  (delay slot)`
becomes
`if $2=0 then`
`  add $1,$2,$3`
	`sub $4,$5,$6`
	`add $1,$2,$3`
	`if $1=0 then`
	`  (delay slot)`
	becomes
	`sub $4,$5,$6`
	`add $1,$2,$3`
	`if $1=0 then`
	`  sub $4,$5,$6`
		`add $1,$2,$3`
		`if $1=0 then`
		`  (delay slot)`
		`or  $7,$8,$9`
		`sub $4,$5,$6`
		becomes
		`add $1,$2,$3`
		`if $1=0 then`
		`  or $7,$8,$9`
		`sub $4,$5,$6`

- **A** is the best choice: it always improves performance (fills the delay slot without duplicating any instruction, reducing instruction count). The branch must not depend on the rescheduled instruction.
- In **B**, the moved instruction may need to be *copied*, increasing instruction count; it must be safe to execute even if the branch is not taken.
- In **C**, the instruction must be safe to execute even if the branch *is* taken.

Three cases for choosing the delay-slot instruction.

A – independent instruction from before the branch (*common*)
Always improves performance when used; the branch must not depend on the rescheduled instruction.

B – instruction from the branch target
Improves performance if the branch is taken; may require duplication; must be safe if the branch is *not* taken.

C – instruction from the fall-through stream
Improves performance if the branch is *not* taken; must be safe if the branch *is* taken.

> **Remark.** The usability of cases B and C is improved by using a **canceling (nullifying) branch**.

3.12.6 Delayed branch: compiler effectiveness

- Fills about 60% of branch delay slots.
- About 80% of instructions placed in delay slots are useful.
- $\Rightarrow$ about 50% ($60\% \times 80\%$) of slots are *usefully* filled.

> **Remark. Downside:** as processors move to deeper pipelines and multiple issue, the branch delay grows and more than one delay slot is needed. Delayed branching has lost popularity relative to more flexible (but more expensive) dynamic approaches, since growth in available transistors has made dynamic branch prediction relatively cheaper.

3.12.7 Evaluating branch alternatives

Proposition 3.16 (Pipeline speedup with branches).

$$\text{Pipeline speedup} = \frac{\text{Pipeline depth}}{1 + \text{Branch frequency} \times \text{Branch penalty}}$$

Example (Comparing branch schemes). Assume: 4% unconditional branches, 6% conditional-untaken, 10% conditional-taken.

Scheduling scheme	Branch penalty	CPI	Speedup v. unpipelined	Speedup v. stall
Stall pipeline	3	1.60	3.1	1.00
Predict not taken	$1\times0.04 + 3\times0.10$	1.34	3.7	1.19
Predict taken	$1\times0.14 + 2\times0.06$	1.26	4.0	1.29
Delayed branch	0.5	1.10	4.5	1.45

3.12.8 Pipeline performance example

Example (CPI of a realistic mix). MIPS instruction mix:

Type	Frequency
Arith/Logic	40%
Load	30% (of which 25% are immediately followed by an instruction using the loaded value)
Store	10%
Branch	20% (of which 45% are taken)

With forwarding and branch-address calculation in the ID stage, using a branch-not-taken scheme (branch penalty = 1 cycle):

$$\text{CPI} = 1 + \underbrace{0.3 \times 0.25 \times 1}_{\text{load-use stalls}} + \underbrace{0.2 \times 0.45 \times 1}_{\text{branch stalls}} = 1 + 0.075 + 0.09 = 1.165$$

3.13 Exercises

Left unresolved for self-study.

Exercise 3.17 (Hazards, NOPs and forwarding). Consider the following instruction sequence, to be executed on the standard 5-stage MIPS pipeline (IF, ID, EX, MEM, WB), assuming registers are written in the first half of a cycle and read in the second half:

```
(1) lw  $1, 40($6)
(2) add $6, $2, $2
(3) sw  $6, 50($1)
(4) add $4, $5, $6
(5) lw  $6, 10($4)
```

Answer the following, showing your reasoning and/or a pipeline diagram for each part:

(A) Identify *all* the data dependencies in the code above (which instruction depends on

which, and through which register).

(B) Assuming the pipeline has *no* forwarding hardware, identify which of the dependencies found in (A) actually cause data hazards (i.e. where a read would happen before the corresponding write in the unmodified pipeline).

(C) Show the code with explicit `NOP` (no-op) instructions inserted so that it executes correctly *without* forwarding hardware.

(D) How many clock cycles does the code from part (C) take to execute?

(E) Now assume the pipeline *does* have full forwarding hardware (forwarding NOPs are inserted only where forwarding cannot resolve the hazard). How many clock cycles does the original code take to execute with forwarding? Identify which specific forwarding paths (EX/MEM → EX, MEM/WB → EX, etc.) are used, and in which cycle.

3.14 Summary

- Pipelining overlaps the execution of multiple instructions.
- With an *ideal* pipeline, CPI = 1, and speedup equals the number of pipeline stages.
- Several factors prevent the ideal speedup from being reached:
  - Not being able to divide the pipeline evenly.
  - The time needed to fill and drain the pipeline.
  - Overhead needed for pipelining (extra latches/control).
  - Structural, data, and control hazards.
- Pipelining is easy when tasks are independent – just overlap them.

$$\text{Speedup} = \frac{\text{Pipeline Depth}}{1 + \text{Pipeline stall CPI}} \times \frac{\text{Clock Cycle Unpipelined}}{\text{Clock Cycle Pipelined}} \qquad \text{(if ideal CPI} = 1\text{)}$$

- **Hazards limit performance:**
  - Structural: need more hardware resources.
  - Data: need forwarding, compiler scheduling.
  - Control: early evaluation & PC computation, delayed branch, branch prediction.
- Increasing pipeline length increases the impact of hazards; pipelining helps instruction *bandwidth*, not *latency*.
- Compilers help reduce the cost of data and control hazards via: load delay-slot scheduling, branch delay-slot scheduling, and branch prediction.